

TUGAS AKHIR

Sistem Rekomendasi Wisata Jember Berbasis *Implicit Feedback* Menggunakan Metode *Hybrid Filtering*



Oleh:
Septian Putra Rachman Hakim
NIM. 2210671037

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2026**

TUGAS AKHIR

Sistem Rekomendasi Wisata Jember Berbasis *Implicit Feedback* Menggunakan Metode *Hybrid Filtering*

Diajukan sebagai salah satu syarat untuk kelulusan
Strata Satu (S-1) Program Studi Sistem Informasi Fakultas Teknik
Universitas Muhammadiyah Jember



Oleh:
Septian Putra Rachman Hakim
2210671037

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

202

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR GAMBAR	4
DAFTAR TABEL	5
DAFTAR LAMPIRAN	6
BAB 1 PENDAHULUAN	7
1.1 Latar Belakang	7
1.2 Rumusan Masalah	9
1.3 Tujuan Penelitian	9
1.4 Manfaat Penelitian	10
1.5 Batasan Masalah.....	10
BAB 2 KAJIAN PUSTAKA.....	12
2.1 Sistem Rekomendasi (<i>Recommender Systems</i>).....	12
2.1.1 <i>Content-Based Filtering</i> (CBF)	12
2.1.2 <i>Collaborative Filtering</i> (CF).....	13
2.1.3 <i>Memory-Based Collaborative Filtering</i>	14
2.1.4 <i>Hybrid Filtering</i>	15
2.2 <i>Implicit Feedback</i>	16
2.3 <i>Arsitektur Transformer</i> (all-MiniLM-L6-v2)	17
2.4 <i>Cosine Similarity</i>	19
2.5 <i>Algoritma K-Nearest Neighbors</i> (KNN).....	20
2.6 <i>Sentence Embedding</i>	21
2.7 <i>Basis Data Vektor (Vector database)</i>	21
2.7.1 <i>Konsep Pencarian Kemiripan (Similarity Search)</i>	22
2.8 <i>SQLAlchemy</i>	22

2.9	<i>K-Fold Cross Validation</i>	23
2.10	Metrik Evaluasi	24
2.10.1	<i>Precision@K</i>	24
2.10.2	<i>Recall@K</i>	25
2.10.3	<i>F1-Score</i>	25
2.11	Penelitian Terdahulu	26
BAB 3	METODOLOGI PENELITIAN	28
3.1	Prosedur Penelitian	28
3.1.1	Identifikasi Masalah & Studi Literatur	29
3.1.2	Tahap Pengumpulan Data	29
3.1.3	Implementasi Sistem Rekomendasi	30
3.1.4	<i>K-Fold Cross Validation</i>	38
3.1.5	Skenario Pengujian	39
3.1.6	Evaluasi Sistem	42
3.2	Jadwal Tabel Penelitian	43
BAB 4	HASIL DAN PEMBAHASAN	45
4.1	Implementasi Sistem	45
4.1.1	Lingkungan Implementasi	45
4.1.2	Deskripsi Data Penelitian	48
4.1.3	Implementasi Fitur Rekomendasi	49
4.1.4	Antarmuka Sistem (<i>User Interface</i>)	50
4.2	Skenario Pengujian	53
4.2.1	Skenario Pengujian <i>Content-Based Filtering</i> (Wisata Serupa)	53
4.2.2	Skenario Pengujian <i>Memory-Based Collaborative Filtering</i> (Rekomendasi <i>Personalize</i>)	54
4.2.3	Skenario Pengujian <i>Hybrid Filtering</i> (<i>Weighted Hybrid</i>)	55

4.3	Hasil Evaluasi dan Pembahasan.....	56
4.3.1	Hasil Evaluasi Skenario <i>Content-Based Filtering</i> (CBF).....	56
4.3.2	Hasil Evaluasi Skenario Memory-Based <i>Collaborative Filtering</i>	57
4.3.3	Hasil Evaluasi Skenario <i>Hybrid Filtering</i>	58
BAB 5	PENUTUP.....	63
5.1	Kesimpulan	63
5.2	Saran.....	64
DAFTAR PUSTAKA		65
LAMPIRAN.....		67

DAFTAR GAMBAR

Gambar 2. 1 Ilustrasi Proses Metode Content Based Filtering	13
Gambar 2. 2 Ilustrasi Proses Metode Collaborative Filtering.....	14
Gambar 2. 3 Arsitektur Transformer Encoder	18
Gambar 2. 2 Knowledge Distillation antara Teacher dan Student Model	19
Gambar 3. 1 Diagram Alir Prosedur Penelitian	28
Gambar 3. 2 Data Destinasi Wisata Jember (destinasi_Processed.csv).....	29
Gambar 3. 3 Simulasi Data Implicit Feedback	30
Gambar 3. 4 Arsitektur AI <i>Pipeline</i> JemberTrip.....	31
Gambar 4. 1 Sistem Backend dan Basis Data JemberTrip.....	47
Gambar 4. 2 Sistem Frontend JemberTrip	48
Gambar 4. 3 Tampilan Rekomendasi Personalize	51
Gambar 4. 4 Tampilan Halaman Yang Mungkin.....	52
Gambar 4. 5 Tampilan Halaman Wisata Serupa.....	52
Gambar 4. 6 Grafik Fluktuasi Metrik Evaluasi	60
Gambar 4. 7 Perbandingan Performa Ketiga Algoritma Rekomendasi	61

DAFTAR TABEL

Tabel 2. 1 Penelitian Terdahulu	26
Tabel 3. 1 Tahap Preprocessing Data Destinasi Wisata.....	32
Tabel 3. 2 Tahap Preprocessing Data <i>Clickstream</i> User.....	32
Tabel 3. 3 Jenis Pemrosesan Berdasarkan Skenario	35
Tabel 3. 4 Skenario Validasi Eksklusi Berdasarkan	36
Tabel 3. 5 <i>Hybrid</i> Filtering.....	37
Tabel 3. 6 Implementasi Panel Antarmuka Keluaran Sistem	38
Tabel 3. 7 Jadwal Penelitian.....	44
Tabel 4. 1 Spesifikasi Perangkat Keras (<i>Hardware</i>).....	45
Tabel 4. 2 Lingkungan Implementasi <i>Backend</i>	46
Tabel 4. 3 Lingkungan Implementasi <i>Frontend</i>	46
Tabel 4. 4 Dataset JemberTrip	48
Tabel 4. 5 Fitur Rekomendasi	49
Tabel 4. 6 Implementasi Antarmuka Fitur Rekomendasi	51
Tabel 4. 7 Rincian Eksekusi Skenario Pengujian <i>Content-Based Filtering</i>	53
Tabel 4. 8 Rincian Eksekusi Skenario Pengujian <i>Memory-Based</i>	54
Tabel 4. 9 Rincian Eksekusi Skenario Pengujian <i>Hybrid Filtering</i>	55
Tabel 4. 10 Hasil Pengujian Parameter K pada <i>Content-Based Filtering</i>	57
Tabel 4. 11 Hasil Pengujian Parameter K pada <i>Collaborative Filtering</i>	58
Tabel 4. 12 Hasil Pengujian Pengujian Parameter K pada <i>Hybrid Filtering</i>	59
Tabel 4. 13 Hasil Pengujian Pembobotan <i>Alpha</i> pada <i>Hybrid Filtering</i>	59
Tabel 4. 14 Ringkasan Peningkatan Performa (CF + <i>Hybrid</i>)	62

DAFTAR LAMPIRAN

Lampiran 1. Source Code Metode Content-Based Filtering	67
Lampiran 2. Source Code Metode Memory-Based Collaborative Filtering	67
Lampiran 3. Source Code Metode Hybrid Filtering	67
Lampiran 4. Source Code Skrip Pengujian Metrik Alpha	68
Lampiran 5. Source Code Skrip Pengujian KNN	69
Lampiran 6. Source Code Skrip Metrik Evaluasi	71
Lampiran 7. Hasil Eksekusi Komputasi Content-Based Filtering	73
Lampiran 8. Hasil Eksekusi Komputasi Memory-Based CF	73
Lampiran 9. Hasil Eksekusi Komputasi Hybrid Filtering.....	73
Lampiran 10. Hasil Eksekusi Pencarian Bobot Alpha pada Hybrid Filtering	74
Lampiran 11. Dataset Implicit Feedback	74

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Sektor pariwisata merupakan salah satu pilar penting dalam perekonomian Indonesia karena kekayaan alam dan budayanya yang melimpah. Berdasarkan data Badan Pusat Statistik (BPS), minat kunjungan wisatawan ke berbagai destinasi terus menunjukkan tren positif setiap tahunnya (Faurina & Sitanggang, 2023). Namun, perkembangan pesat ini menimbulkan tantangan baru berupa kelebihan informasi (*information overload*), di mana wisatawan sering kali mengalami kesulitan dalam menemukan destinasi yang benar-benar sesuai dengan preferensi pribadi mereka karena banyaknya pilihan yang tersedia (Natasya, 2023).

Dalam era digital, sistem rekomendasi menjadi elemen vital untuk membantu wisatawan menavigasi informasi destinasi. Banyak sistem saat ini menerapkan metode *Content-Based Filtering* (CBF) yang bekerja dengan mengukur kemiripan antar destinasi menggunakan pendekatan leksikal seperti TF-IDF (Oyadila dkk., 2025). Namun, pendekatan konvensional tersebut memiliki keterbatasan signifikan, yaitu ketidakmampuan memahami hubungan semantik secara mendalam antar atribut destinasi. Akibatnya, rekomendasi yang diberikan sering kali hanya terpaku pada kesamaan kata kunci secara literal dan gagal menangkap konteks deskriptif yang lebih kaya.

Kabupaten Jember, yang secara geografis diuntungkan oleh bentang alam pegunungan hingga pesisir pantai selatan, memiliki potensi pariwisata yang masif dan bernilai strategis bagi pertumbuhan ekonomi daerah. Jember tidak hanya menawarkan keindahan wisata bahari, tetapi juga memiliki kekayaan agrowisata unggulan, ekowisata, rekreasi buatan, hingga situs sejarah dan budaya. Dengan puluhan titik destinasi yang tersebar luas di berbagai kecamatan, Jember sejatinya memiliki daya tarik yang sangat besar untuk menjadi pusat kunjungan wisatawan domestik maupun mancanegara. Sayangnya, besarnya potensi wilayah ini belum sejalan dengan meratanya distribusi informasi secara digital. Wisatawan sering kali mengalami *information overload* atau kebingungan akibat banyaknya pilihan, sehingga kunjungan wisata hanya terpusat pada beberapa destinasi populer saja. Sementara itu, banyak destinasi potensial berstatus *hidden gems* kurang terekspos

akibat minimnya data interaksi pengguna di dunia maya (Cholil dkk., 2023). Permasalahan ini diperparah oleh sebagian besar sistem rekomendasi wisata saat ini yang masih sangat mengandalkan data Explicit Feedback berupa pemberian rating atau ulasan (Dwiputriane dkk., 2022). Pendekatan ini sering memunculkan kendala Cold start Problem, di mana destinasi wisata baru atau hidden gems di Jember akan sangat sulit direkomendasikan kepada wisatawan karena belum memiliki rating, serta rendahnya partisipasi pengguna dalam memberikan penilaian secara manual.

Berdasarkan permasalahan tersebut, terdapat kesenjangan (*research gap*) yang signifikan pada penelitian-penelitian terdahulu. Meskipun teknologi *semantic embedding* seperti BERT telah mulai diterapkan untuk memahami konteks teks pariwisata, sebagian besar implementasinya masih berfokus pada data eksplisit dan mengabaikan interaksi dinamis pengguna (Channarong dkk., 2022). Sebaliknya, tinjauan literatur sistematis terbaru menunjukkan bahwa sistem yang hanya berfokus pada data interaksi pengguna (*Implicit Feedback*) seringkali kesulitan untuk mengidentifikasi hubungan kemiripan karakteristik antar item secara mendalam (Gunathilaka dkk., 2025). Hingga saat ini, masih sangat minim penelitian yang berhasil mengintegrasikan pemahaman semantik teks (*Semantic embedding*) dengan interaksi perilaku pengguna secara otomatis (*Implicit Feedback*) di dalam satu ekosistem komputasi yang ringan dan mampu berjalan secara *real-time*.

Untuk mengisi kesenjangan tersebut, penelitian ini mengusulkan JemberTrip sebuah sistem rekomendasi cerdas yang menerapkan Metode *Hybrid Filtering* berbasis *Implicit Feedback*. Berbeda dengan sistem konvensional, JemberTrip menangkap sinyal ketertarikan pengguna secara otomatis melalui rekam jejak riwayat klik (*clickstream*) tanpa memerlukan input *rating* manual. Guna mengatasi masalah pemahaman konteks dan *Cold start Problem*, sistem ini mengintegrasikan model *Transformer* all-MiniLM-L6-v2 untuk mengubah deskripsi tekstual destinasi menjadi vektor representasi semantik berdimensi 384. Penggunaan model ini terbukti memiliki efisiensi tinggi dengan kecepatan inferensi 5,3 kali lebih cepat dibanding model besar seperti BERT, sehingga sangat ideal untuk kebutuhan sistem *real-time* (Wardani dkk., 2025).

Sistem ini secara khusus dirancang dengan tiga luaran utama, yaitu Wisata Serupa (pendekatan *Item-to-Item*), Rekomendasi *Personalize* (pendekatan *User-to-Item*), dan Mungkin Yang Anda Sukai (pendekatan *Weighted Hybrid*). Fitur-fitur ini dipersonalisasi berdasarkan profil riwayat eksplorasi pengguna melalui mekanisme riwayat klik (*clickstream*). Dengan kombinasi pendekatan tersebut serta penerapan teknologi *semantic embedding*, penelitian ini diharapkan mampu memberikan kontribusi nyata dalam optimalisasi promosi wisata lokal di Kabupaten Jember, serta meningkatkan pengalaman navigasi wisatawan melalui hasil rekomendasi yang jauh lebih kontekstual, akurat, dan adaptif.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Content Based Filtering*?
2. Bagaimana tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Memory-Based Collaborative Filtering*?
3. Bagaimana tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Hybrid Filtering*?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah disusun, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Mengukur tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Content Based Filtering*
2. Mengukur tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Memory-Based Collaborative Filtering*
3. Mengukur tingkat akurasi dari sistem rekomendasi wisata jember berbasis implicit feedback menggunakan metode *Hybrid Filtering*?

1.4 Manfaat Penelitian

1. Bagi Pemerintah Kabupaten Jember

Penelitian ini diharapkan memberikan manfaat nyata dalam pengembangan sistem informasi pariwisata berbasis kecerdasan buatan, khususnya dalam konteks daerah Kabupaten Jember yang memiliki potensi wisata besar namun belum tereksplorasi secara optimal.

2. Bagi Pengguna (Wisatawan)

Penelitian ini diharapkan dapat memberikan pengalaman eksplorasi yang lebih efisien dan personal dalam menemukan destinasi di Kabupaten Jember, sehingga pengguna terhindar dari kendala kelebihan informasi (*information overload*) dan mendapatkan rekomendasi tempat wisata yang benar-benar sesuai dengan minat serta profil perilaku mereka secara otomatis tanpa perlu melakukan penilaian manual.

3. Bagi Peneliti

Sebagai sarana untuk mengimplementasikan dan mengembangkan ilmu pengetahuan yang telah diperoleh selama perkuliahan, khususnya dalam bidang Kecerdasan Buatan (*Artificial Intelligence*) terkait penerapan metode *Hybrid Filtering* dan data *Implicit Feedback*. Selain itu, penelitian ini disusun sebagai salah satu syarat wajib untuk menyelesaikan pendidikan Strata Satu (S-1) dan meraih gelar Sarjana Komputer (S.Kom) pada Program Studi Sistem Informasi, Fakultas Teknik, Universitas Muhammadiyah Jember.

1.5 Batasan Masalah

1. Objek penelitian mencakup deskripsi tentang destinasi wisata berjumlah 56 yang berada di dalam wilayah administratif Kabupaten Jember. Data ini bersumber dari portal resmi Dinas Pariwisata dan Kebudayaan Kabupaten Jember (jembertourism.jemberkab.go.id/peta-wisata/) yang diakses dan dikurasi pada tanggal 07 Januari 2026.
2. Data yang digunakan untuk personalisasi adalah *Implicit Feedback* yang hanya dibatasi pada riwayat klik (*clickstream*) pengguna terhadap destinasi wisata di dalam aplikasi JemberTrip.

3. Menggunakan model pre-trained all-MiniLM-L6-v2 (Encoder-only *Transformer*) untuk ekstraksi fitur semantik tanpa melakukan proses pelatihan ulang atau fine-tuning model.
4. Penerapan algoritma *K-Nearest Neighbors* (KNN) dalam sistem dibatasi pengujiannya hingga maksimal $K=30$. Hasil penentuan parameter terbaik membatasi sistem pada pencarian 10 *nearest neighbors* (Top-10) untuk fitur wisata serupa (*Content-Based Filtering*), dan 30 *nearest neighbors* (Top-30) untuk fitur rekomendasi personalisasi (*Memory-Based Collaborative Filtering*). Sementara untuk metode *Hybrid Filtering*, penggabungan skor dibatasi menggunakan metode *Weighted Hybrid* dengan parameter bobot $\alpha = 0.6$ (60% CF, 40% CBF).
5. Pengujian performa sistem rekomendasi dibatasi pada penghitungan nilai $Precision@K$ ($P@K$), $Recall@K$ ($R@K$) dan $F1-Score$ untuk mengukur kualitas daftar rekomendasi yang dihasilkan. Nilai K ditetapkan sebesar 6 (@6) yang disesuaikan dengan batasan jumlah maksimal kartu rekomendasi yang ditampilkan pada antarmuka pengguna (UI) website JemberTrip.

BAB 2 KAJIAN PUSTAKA

2.1 Sistem Rekomendasi (*Recommender Systems*)

Sistem rekomendasi (*recommender system*) merupakan salah satu cabang penting dari sistem informasi cerdas yang berfungsi membantu pengguna dalam menemukan informasi atau item yang relevan berdasarkan preferensi dan perilaku mereka (Gunathilaka dkk., 2025). Sistem rekomendasi mengatasi masalah *information overload*, yaitu kondisi ketika jumlah informasi yang tersedia terlalu banyak untuk diproses secara manual oleh pengguna (Khumaidi dkk., 2025). Dalam konteks pariwisata, sistem rekomendasi digunakan untuk memberikan saran destinasi wisata yang sesuai dengan kebutuhan, minat, dan karakteristik wisatawan.

Sistem rekomendasi bekerja dengan cara menganalisis hubungan antara pengguna (*user*) dan item (dalam hal ini objek wisata) untuk menghasilkan saran yang dipersonalisasi. Secara umum, sistem rekomendasi terbagi menjadi tiga kategori utama, yaitu *Content-Based Filtering* (CBF), *Collaborative Filtering* (CF), dan *Hybrid Filtering*, di mana masing-masing memiliki pendekatan dan keunggulan tersendiri (Gunathilaka dkk., 2025).

2.1.1 *Content-Based Filtering* (CBF)

Pendekatan *Content-Based Filtering* (CBF) merupakan salah satu arsitektur utama dalam sistem rekomendasi yang bekerja dengan cara menganalisis derajat kemiripan antara item yang direpresentasikan, dengan profil preferensi atau riwayat item yang pernah dilihat oleh pengguna di masa lalu (Gunathilaka dkk., 2025). Secara teoritis, algoritma ini bertugas untuk merekomendasikan sekumpulan entitas baru kepada seorang subjek berdasarkan derajat kemiripan atribut konten yang dikandung oleh entitas tersebut, dengan rekam jejak entitas yang pernah direspons positif oleh subjek di masa lalu. Pendekatan ini murni mengeksplorasi fitur internal yang melekat pada objek itu sendiri, seperti ekstraksi teks, *metadata*, maupun kategori, guna membangun sebuah profil representasi informasi (Gunathilaka dkk., 2025).



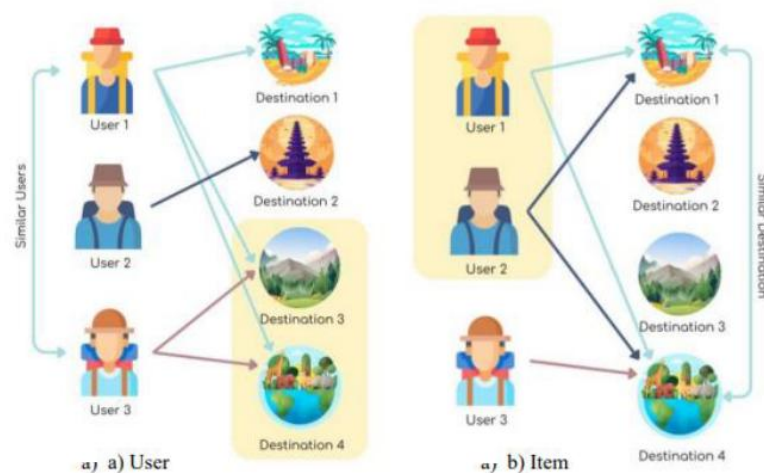
Gambar 2. 1 Ilustrasi Proses Metode *Content Based Filtering*

(Sumber: Faurina & Sitanggang, 2023)

Dalam literatur pengolahan bahasa alami tingkat lanjut, proses pembentukan representasi fitur pada algoritma CBF tidak lagi dibatasi oleh analisis leksikal frekuensi kata seperti metode TF-IDF, melainkan telah memanfaatkan teknik proyeksi ruang vektor (*vector space model*) tingkat tinggi. Teknik seperti *Word Embeddings* maupun arsitektur *Sentence Embeddings* (seperti Transformer) digunakan untuk mentransformasikan narasi teks tidak terstruktur menjadi deretan matriks vektor numerik, sehingga relasi makna semantik antar-entitas dapat dikalkulasi secara komputasional (Oyadila dkk., 2025; Romadhani dkk., 2025).

2.1.2 *Collaborative Filtering* (CF)

Dalam literatur sistem penemuan informasi, *Collaborative Filtering* (CF) diakui sebagai salah satu paradigma paling dominan yang beroperasi dengan mengeksplorasi rekam jejak perilaku dan interaksi pengguna di masa lalu guna memproyeksikan preferensi di masa depan (Gunathilaka dkk., 2025). Secara konseptual, pendekatan ini didasari oleh asumsi sosiologis matematis: apabila entitas Pengguna A dan Pengguna B memiliki pola ketertarikan yang identik terhadap sekumpulan objek di masa lalu, maka probabilitas Pengguna A untuk menyukai objek baru yang dievaluasi positif oleh Pengguna B bernilai sangat tinggi. Berbeda dengan CBF yang menuntut ketersediaan ekstraksi fitur domain secara intensif, metode CF murni mengandalkan agregasi data interaksi (*feedback*) lintas pengguna untuk menghasilkan komputasi prediksi yang lebih akurat secara personal (Dwiputriane dkk., 2022).



Gambar 2. 2 Ilustrasi Proses Metode *Collaborative Filtering*
(Sumber: Faurina & Sitanggang, 2023)

2.1.3 *Memory-Based Collaborative Filtering*

Collaborative Filtering (CF) merupakan teknik rekomendasi yang bekerja berdasarkan pola interaksi antar pengguna dan item (Gunathilaka dkk., 2025). Metode ini terbagi menjadi dua kategori utama, yaitu *Model-Based* dan *Memory-Based*. *Memory-Based Collaborative Filtering* adalah pendekatan yang menggunakan seluruh data interaksi user-item yang tersimpan dalam sistem secara langsung untuk menghasilkan prediksi tanpa melalui proses pelatihan model yang kompleks (George & Lal, 2022). Pendekatan *Memory-Based* biasanya mengandalkan perhitungan kemiripan (*similarity*) antar pengguna (*User-Based*) atau antar item (*Item-Based*) (Gunathilaka dkk., 2025).

Keunggulan utama dari *Memory-Based Collaborative Filtering* adalah kemampuannya dalam memberikan hasil yang transparan dan mudah diperbarui seiring bertambahnya data interaksi pengguna. Namun, tantangan utama dari metode ini adalah masalah *data sparsity* dan *cold start* (Dwiputriane dkk., 2022). Dalam penerapannya, metode ini mengandalkan perhitungan kesamaan (*similarity*) untuk menentukan seberapa dekat preferensi seorang pengguna dengan item-item yang tersedia. Terdapat beberapa algoritma yang umum digunakan dalam pendekatan ini, seperti *K-Nearest Neighbors* (KNN) untuk mencari *nearest neighbors* terdekat dan *Cosine similarity* untuk mengukur sudut antar vektor interaksi (Sovina dkk., 2024; Suwitno dkk., 2025).

2.1.4 Hybrid Filtering

Metode *Hybrid Filtering* merupakan gabungan antara *Content-Based Filtering* (CBF) dan *Collaborative Filtering* (CF) yang dirancang untuk mengatasi kekurangan masing-masing pendekatan tunggal (Channarong dkk., 2022). Sistem *Hybrid* mampu memanfaatkan informasi konten sekaligus perilaku pengguna secara bersamaan, sehingga menghasilkan rekomendasi yang lebih akurat dan kontekstual (Khumaidi dkk., 2025). Perkembangan sistem rekomendasi di bidang pariwisata menunjukkan tren penggunaan pendekatan semantik dan kecerdasan buatan untuk memahami makna teks secara lebih mendalam (Romadhani dkk., 2025). Sistem tradisional yang hanya bergantung pada leksikal kata kunci kini dianggap kurang efektif karena tidak mampu menangkap konteks dan sinonim dalam deskripsi wisata (Channarong dkk., 2022).

Salah satu skema penggabungan yang lazim digunakan dalam sistem *Hybrid* adalah *Weighted Hybrid* (hibrida berbasis pembobotan). Namun, sebelum agregasi pembobotan dapat dilakukan, sistem perlu menyelaraskan perbedaan distribusi nilai (skala) dari masing-masing algoritma tunggal. Oleh karena itu, dilakukan normalisasi pada skor prediksi dengan menggunakan *Min-Max Scaling*. *Min-Max normalization* merupakan metode normalisasi dengan melakukan transformasi linier terhadap data asli sehingga menghasilkan keseimbangan nilai perbandingan antar data saat sebelum dan sesudah proses (Faurina & Sitanggang, 2023).

Metode transformasi ini mereduksi skala probabilitas ke dalam rentang rasio 0 hingga 1 dengan menggunakan persamaan berikut (Faurina & Sitanggang, 2023):

$$\text{Norm}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

Keterangan:

- a. $\text{Norm}(x)$: nilai hasil normalisasi.
- b. x : nilai asli.
- c. $x_{\min}$: nilai minimum data.
- d. $x_{\max}$: nilai maksimum data.

Setelah metrik probabilitas dari seluruh algoritma berada pada skala yang seimbang, probabilitas akhir dihitung melalui persamaan linier yang melibatkan parameter pembobot (weight factor), yang sering disimbolkan dengan α (*Alpha*).

Parameter ini bertindak sebagai rasio pengendali yang menentukan seberapa besar dominasi suatu algoritma di dalam kalkulasi akhir. Formulasi dari *Weighted Hybrid* ini ditunjukkan pada Persamaan berikut (Rachmaniar dkk., 2025):

$$S_H = \alpha S_{CF} + (1-\alpha) S_{CBF} \quad (2)$$

Keterangan:

- a. S_H : skor rekomendasi hasil metode *Hybrid*.
- b. S_{CF} : skor rekomendasi yang dihasilkan oleh metode *Collaborative Filtering*.
- c. S_{CBF} : skor rekomendasi yang dihasilkan oleh metode *Content-Based Filtering*.
- d. α : bobot metode *Collaborative Filtering*, dengan $0 \leq \alpha \leq 1$.

Melalui injeksi parameter α tersebut, sistem memiliki fleksibilitas matematis untuk beradaptasi. Sebagai contoh, dengan menggeser nilai α , algoritma dapat memberikan porsi pembobotan yang lebih besar pada hasil kemiripan konten (CBF) apabila data interaksi pengguna (CF) terindikasi sangat keropos atau tidak memadai. Mekanisme inilah yang menjadikan algoritma *Hybrid* sangat tangguh dalam menangani masalah kekosongan informasi awal (Rachmaniar dkk., 2025).

2.2 *Implicit Feedback*

Dalam sistem rekomendasi, feedback atau umpan balik dari pengguna merupakan data utama yang digunakan untuk mempelajari preferensi mereka (Dwiputriane dkk., 2022). Secara umum, terdapat dua jenis umpan balik, yaitu *Explicit Feedback* dan *Implicit Feedback* (Dwiputriane dkk., 2022). *Explicit Feedback* adalah data yang diberikan secara sadar oleh pengguna, seperti pemberian nilai *rating* (1-5 bintang) atau ulasan teks. Namun, pendekatan ini memiliki kelemahan berupa rendahnya tingkat partisipasi pengguna dan munculnya masalah *data sparsity* karena tidak semua pengguna bersedia memberikan penilaian secara manual (Gunathilaka dkk., 2025).

Implicit Feedback hadir sebagai solusi dengan merekam aktivitas atau perilaku pengguna secara otomatis selama berinteraksi dengan sistem. Data ini dapat berupa riwayat klik (*clickstream*), durasi melihat halaman, hingga riwayat pencarian (Esmeli dkk., 2023; Winardy dkk., 2022).

2.3 Arsitektur *Transformer* (all-MiniLM-L6-v2)

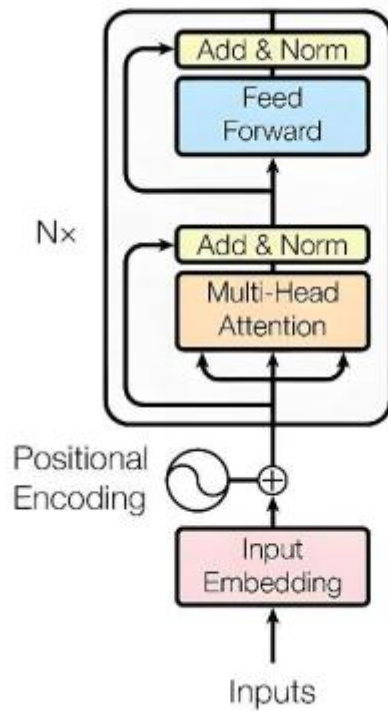
Model all-MiniLM-L6-v2 merupakan varian arsitektur *Transformer* yang dioptimasi untuk menghasilkan *word embeddings* secara efisien. Dasar dari model ini adalah mekanisme *self-attention* yang diperkenalkan oleh Vaswani dkk. (2017), yang dirumuskan sebagai berikut (Vaswani dkk., 2017):

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

Keterangan:

- a. Q (*Query*): vektor representasi dari kata (token) target yang sedang menjadi fokus utama pada saat itu.
- b. K (*Key*): Merupakan vektor representasi dari seluruh kata lain yang ada di dalam urutan masukan.
- c. V (*Value*): vektor yang menyimpan makna atau konten informasi aktual dari kata tersebut.
- d. T (*Transpose*): simbol operasi matematika putaran matriks (*transpose*) yang diterapkan pada matriks K .
- e. $\sqrt{d_k}$ (*Scaling Faktor*): Merupakan nilai akar kuadrat dari dimensi vektor *key* (d_k)

Mekanisme ini memungkinkan model memahami hubungan kontekstual antar kata secara paralel dalam suatu kalimat. Model ini menggunakan arsitektur encoder-only dengan 6 lapisan (L6) dan dimensi hidden sebesar 384 (H384). Arsitektur encoder *Transformer* secara umum dapat dilihat pada Gambar 2.3, yang menjadi dasar pengembangan model all-MiniLM-L6-v2.



Gambar 2. 3 Arsitektur *Transformer* Encoder

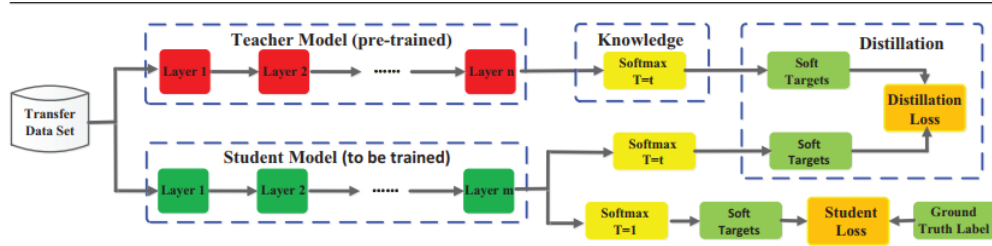
(Sumber: Wardani, 2025)

Keunggulan utama MiniLM terletak pada proses pembuatannya melalui *knowledge distillation*, yaitu pelatihan model kecil (*student*) untuk meniru representasi model besar (*teacher*) (Gou dkk., 2022). Berbeda dengan distilasi berbasis output, MiniLM secara khusus mentransfer informasi pada mekanisme *self-attention*.

Distribusi perhatian (*attention distribution*) diperoleh dari hasil normalisasi *softmax* pada matriks perhatian. Untuk menyelaraskan distribusi tersebut antara model *teacher* dan *student*, digunakan fungsi *Kullback-Leibler Divergence* (KLD) sebagai berikut (Gou dkk., 2022):

$$L_{KD} = \text{KL}(Q^T \parallel Q^S) \quad (4)$$

Di mana Q^T dan Q^S masing-masing merepresentasikan distribusi *self-attention* dari model *teacher* dan *student*. Melalui pendekatan ini, MiniLM mampu mempertahankan performa yang kompetitif dengan model besar seperti BERT-base, sekaligus meningkatkan efisiensi komputasi secara signifikan (Gou dkk., 2022). Ilustrasi proses distilasi ditunjukkan pada Gambar 2.2.



Gambar 2. 4 Knowledge Distillation antara Teacher dan Student Model

(Sumber: Gou, 2022)

2.4 Cosine Similarity

Cosine similarity merupakan metode yang umum digunakan untuk mengukur tingkat kemiripan antara dua vektor dalam ruang multidimensi, khususnya dalam sistem rekomendasi berbasis teks (Pratiwi dkk., 2024). Dalam *Content-Based Filtering*, *cosine similarity* digunakan untuk membandingkan profil pengguna dan item, metode ini menghitung nilai kosinus dari sudut antara dua vektor, sehingga mampu merepresentasikan kesamaan berdasarkan arah vektor, bukan panjangnya dari sebuah deskripsi (Oyadila dkk., 2025). Berbeda dengan *Euclidean Distance* yang mengukur jarak lurus antar titik, *Cosine similarity* mengukur sudut kosinus antara dua vektor. Hal ini membuat metrik tersebut sangat efektif untuk membandingkan kemiripan teks karena tidak dipengaruhi oleh panjang pendeknya deskripsi, melainkan pada orientasi atau arah maknanya. Untuk menentukan skor kemiripan item, dilakukan perhitungan menggunakan rumus berikut (Oyadila dkk., 2025):

$$\text{Similarity}(A,B)=\cos(\theta)=\frac{A \cdot B}{|A||B|}=\frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (5)$$

Keterangan:

- e. $A \cdot B$: Hasil perkalian titik (*dot product*) antar vektor.
- f. $|A|$ dan $|B|$: Panjang atau magnitudo dari masing-masing vektor.
- g. n : Dimensi vektor (dalam penelitian ini berjumlah 384).
- h. i : Indeks atau urutan dimensi vektor yang sedang dihitung (berjalan dari 1 hingga n)

Nilai *cosine similarity* secara teoritis berada pada rentang -1 hingga 1, namun dalam konteks representasi teks umumnya berada pada rentang 0 hingga 1. Jika nilai mendekati 1, maka sudut antar vektor semakin kecil (θ mendekati 0°),

yang berarti kedua item memiliki kemiripan semantik yang sangat tinggi. Sebaliknya, jika nilai mendekati 0, kedua item dianggap tidak memiliki kemiripan konteks. Hasil inilah yang menjadi dasar bagi sistem untuk mengurutkan daftar item yang memiliki kemiripan (Faurina & Sitanggang, 2023).

2.5 Algoritma *K-Nearest Neighbors* (KNN)

KNN atau *K-Nearest Neighbors*, merupakan satu dari banyak algoritma klasifikasi yang paling banyak digunakan dan dikenal karena kesederhanaannya dan kemudahan implementasinya (Firmansyah dkk., 2024). Algoritma *K-Nearest Neighbors* (KNN) memilih K item terdekat dari suatu item atau profil pengguna berdasarkan nilai kemiripan yang telah dihitung (Gunathilaka dkk., 2025). Cara kerja KNN didasarkan pada prinsip kedekatan. Ketika sebuah data uji baru masuk, KNN akan mencari K *nearest neighbors* terdekatnya dalam ruang fitur data pelatihan. Nilai K ditentukan sebelumnya dan mewakili jumlah *nearest neighbors* yang akan dipertimbangkan. *nearest neighbors* ini dipilih berdasarkan jarak mereka ke data uji, dengan jarak yang lebih kecil menunjukkan kedekatan yang lebih besar. Setelah *nearest neighbors* terdekat ditemukan, KNN akan memprediksi kelas data uji berdasarkan kelas mayoritas di antara *nearest neighbors* tersebut. Dari itu, jika sebagian besar *nearest neighbors* terdekat termasuk dalam kelas tertentu, maka data uji juga akan diklasifikasikan ke dalam kelas tersebut (Firmansyah dkk., 2024). Perhitungan jarak pada KNN menggunakan rumus *Euclidian Distance* yaitu (Maskuri dkk., 2022):

$$d_i = \sqrt{\sum_{i=1}^k (a_i - b_i)^2} \quad (6)$$

Keterangan:

- a. d_i : Jarak euclidean
- b. a_i : Nilai atribut ke- i pada data uji.
- c. b_i : Nilai atribut ke- i pada data latih.
- d. k : Jumlah dimensi atau atribut data.

Langkah – langkah dalam algoritma KNN secara umum yaitu (Maskuri dkk., 2022):

- a) Menentukan nilai k , dengan rumus $k = \sqrt{n}$.

- b) Melakukan perhitungan nilai jarak (*Euclidean Distance*) terhadap masing-masing objek data yang diberikan. Rumus untuk menghitung *Euclidean Distance* dapat dilihat pada persamaan 4.

$$d_i = \sqrt{(x_{ki} - x_{kj})^2 + (x_{ki} - x_{kj})^2 + \dots + (x_{ki} - x_{kj})^2} \quad (7)$$

- c) Melakukan pengelompokkan data sesuai dengan perhitungan jarak (*Euclidean Distance*)
- d) Melakukan pengelompokkan data sesuai dengan nilai *nearest neighbors* terdekat atau berdasarkan data yang mempunyai jarak Euclidean terkecil.
- e) Memilih nilai mayoritas dari nearest neighbors terdekat sebagai hasil klasifikasi.

2.6 *Sentence Embedding*

Sentence embedding merupakan teknik dalam *Natural Language Processing* (NLP) yang berfungsi untuk merepresentasikan sebuah kalimat atau teks ke dalam bentuk vektor numerik berdimensi tinggi (Sutriawan dkk., 2025). Berbeda dengan teknik representasi teks tradisional seperti *Bag-of-Words* atau TF-IDF yang hanya menghitung frekuensi kemunculan kata secara literal, *sentence embedding* mampu menangkap makna semantik dan konteks dari suatu kalimat secara utuh.

2.7 *Basis Data Vektor (Vector database)*

Basis data vektor (*vector database*) merupakan sistem penyimpanan data yang dirancang khusus untuk menangani data dalam bentuk vektor multidimensi. Berbeda dengan basis data relasional tradisional yang melakukan pencarian berbasis kecocokan kata kunci (*keyword matching*), basis data vektor melakukan pencarian berdasarkan kedekatan atau kemiripan antar vektor (*similarity search*) dalam ruang semantik. Pendekatan ini sangat relevan dalam aplikasi kecerdasan buatan, terutama pada sistem yang menggunakan *embedding* untuk merepresentasikan makna data (Nangunori, 2024).

Perkembangan basis data vektor didorong oleh meningkatnya kebutuhan dalam pengolahan data tidak terstruktur seperti teks, gambar, dan audio. Dalam konteks ini, setiap data diubah menjadi representasi numerik berdimensi tinggi sehingga dapat dibandingkan secara matematis. Basis data vektor mampu

menangani kompleksitas ini dengan menyediakan mekanisme *indexing* dan pencarian yang efisien, bahkan pada skala data yang besar (Nangunori, 2024). Dalam sistem rekomendasi modern, basis data vektor berperan sebagai komponen utama dalam mendukung pencarian berbasis makna (*semantic search*), sehingga mampu menghasilkan rekomendasi yang lebih relevan dibandingkan pendekatan berbasis kata kunci.

2.7.1 Konsep Pencarian Kemiripan (*Similarity Search*)

Similarity search merupakan mekanisme utama dalam basis data vektor yang digunakan untuk menemukan item yang memiliki kemiripan tertinggi terhadap suatu *query*. Dalam pendekatan ini, baik data maupun *query* direpresentasikan dalam bentuk vektor numerik, kemudian dibandingkan menggunakan metrik tertentu seperti *cosine similarity*, *Euclidean Distance*, atau *inner product*. Dalam sistem rekomendasi, proses ini dilakukan dengan menghitung tingkat kemiripan antara vektor preferensi pengguna dan vektor item yang tersimpan di dalam basis data. Semakin tinggi nilai kemiripan yang dihasilkan, maka semakin relevan item tersebut terhadap kebutuhan pengguna (Nangunori, 2024). Pendekatan *similarity search* memungkinkan sistem untuk memahami hubungan semantik antar data, sehingga mampu mengatasi keterbatasan metode tradisional yang hanya bergantung pada kecocokan kata. Hal ini sangat penting dalam konteks pariwisata, di mana deskripsi destinasi sering kali memiliki makna yang serupa meskipun menggunakan kata yang berbeda (Nangunori, 2024).

2.8 SQLAlchemy

SQLAlchemy merupakan salah satu kerangka kerja *Object-Relational Mapping* (ORM) pada bahasa pemrograman Python yang berfungsi menyediakan lapisan abstraksi antara kode aplikasi dan sistem basis data (Keogh, 2025). Tujuan utama dari penggunaan teknologi ORM adalah untuk mengatasi tantangan struktural yang secara teoritis dikenal dengan istilah *Object-Relational Impedance Mismatch*. Ketidakcocokan ini muncul karena paradigma pemrograman berorientasi objek mengelola data melalui hierarki kelas, pewarisan, dan referensi, sedangkan basis data relasional menyimpan data menggunakan struktur tabel yang datar, kunci tamu (*foreign keys*), dan logika himpunan (Keogh, 2025). Dengan menerapkan lapisan abstraksi ORM, pengembang dapat memanipulasi dan

berinteraksi dengan rekaman basis data seolah-olah data tersebut adalah objek asli di dalam bahasa pemrograman, sehingga meminimalisir penulisan kode SQL mentah secara berulang (Hermanto dkk., 2025).

Dalam menjembatani perbedaan antara kedua representasi data tersebut, SQLAlchemy bekerja dengan cara menerjemahkan struktur objek ke dalam skema relasional secara otomatis di belakang layar (Keogh, 2025). Pusat operasional dari kerangka kerja ORM ini terletak pada penggunaan konsep *Session*, yakni sebuah komponen *runtime* yang merangkum pengelolaan koneksi ke basis data, pembuatan *query*, serta penyimpanan sementara (*cache*) dari instansiasi objek. Pendekatan ini sangat mendukung penerapan prinsip pemisahan fokus (*separation of concerns*) antara logika bisnis aplikasi dengan mekanisme akses data, yang pada akhirnya berkontribusi pada peningkatan produktivitas pengembang serta kemudahan dalam pemeliharaan kode (Hermanto dkk., 2025). Meskipun menawarkan kemudahan pengembangan yang terstruktur, proses penerjemahan melalui lapisan abstraksi ORM ini pada dasarnya juga memperkenalkan beban komputasi dan memori tambahan (*overhead*) pada sistem.

2.9 *K-Fold Cross Validation*

K-Fold Cross Validation merupakan salah satu teknik validasi statistik yang digunakan untuk mengevaluasi kinerja model atau algoritma dengan cara membagi dataset menjadi sejumlah K bagian (*folds*) yang sama besar (Esmeli dkk., 2023). Dalam setiap iterasi pengujian, satu bagian akan berfungsi sebagai data uji (*test set*), sementara $K - 1$ bagian lainnya bertindak sebagai data pendukung atau acuan. Prosedur ini diulang sebanyak K kali sehingga setiap data dalam dataset pernah digunakan sebagai data uji tepat satu kali.

Merujuk pada penelitian Esmeli dkk. (2023), penggunaan *10-fold cross-validation* diimplementasikan untuk menjamin evaluasi performa yang reliabel dan kuat (*robust*) pada sistem rekomendasi berbasis *Implicit Feedback*. Hasil evaluasi akhir yang dilaporkan merupakan nilai rata-rata dari seluruh putaran validasi tersebut, sehingga mampu memberikan gambaran performa sistem yang lebih stabil dan objektif dibandingkan hanya menggunakan satu kali pembagian data (*simple split*).

2.10 Metrik Evaluasi

Evaluasi sistem merupakan komponen kritis dalam siklus pengembangan sistem informasi, terutama pada sistem yang mengimplementasikan kecerdasan buatan (*Artificial Intelligence*) seperti sistem rekomendasi (Wang & Chen, 2026). Evaluasi sistem berfungsi untuk memvalidasi apakah algoritma yang diusulkan memiliki efektivitas yang signifikan dalam mengatasi permasalahan *information overload* atau hanya memberikan hasil rekomendasi secara acak (*random choice*) (Esmeli dkk., 2023).

Dalam sistem rekomendasi, terdapat dua cara utama untuk mengukur performa, yaitu berbasis prediksi nilai (*Rating/Explicit*) dan berbasis peringkat (*Ranking/Implicit*) (Esmeli dkk., 2023). Metrik menggunakan tiga metrik utama, yaitu:

2.10.1 *Precision@K*

Precision@K mengukur tingkat ketepatan sistem dengan menghitung proporsi item yang benar-benar relevan dari total K item yang direkomendasikan (Esmeli dkk., 2023). Metrik ini sangat penting dari sudut pandang pengguna, karena pengguna biasanya hanya memperhatikan beberapa hasil teratas saja. Secara matematis, *Precision@K* dirumuskan sebagai berikut (Esmeli dkk., 2023):

$$Precision@K = \frac{|RecommendedItems \cap GroundTruth|}{|RecommendedItems|} \quad (8)$$

Keterangan:

- $|RecommendedItems \cap GroundTruth|$: Jumlah destinasi yang muncul di daftar rekomendasi yang memiliki kategori sesuai dengan *Ground Truth* (kunci jawaban) .
- $|RecommendedItems|$: Total jumlah destinasi yang ditampilkan sistem (nilai K).

Nilai $P@K$ berada dalam rentang $[0, 1]$, di mana nilai 1 mengindikasikan bahwa seluruh K destinasi wisata yang direkomendasikan sistem adalah relevan dengan minat pengguna. Metrik ini sangat krusial karena berorientasi pada kualitas hasil pencarian dari perspektif wisatawan (Esmeli dkk., 2023).

2.10.2 *Recall@K*

Recall@K mengukur kemampuan sistem dalam menemukan kembali seluruh item yang relevan dari total dataset yang ada sesuai dengan preferensi pengguna (Esmeli dkk., 2023). Metrik ini menunjukkan seberapa lengkap sistem dalam mencakup item-item yang seharusnya disukai pengguna. Rumus untuk *Recall@K* adalah (Esmeli dkk., 2023):

$$Recall@K = \frac{|Recommended\ Items \cap Ground\ Truth|}{|Ground\ Truth|} \quad (9)$$

Keterangan:

- $|RecommendedItems \cap GroundTruth|$: Jumlah destinasi relevan yang berhasil muncul dalam daftar rekomendasi sistem.
- $|Ground Truth|$: Total seluruh destinasi yang memiliki kategori serupa di dalam dataset.

Recall@K mengukur kemampuan sistem dalam menemukan kembali seluruh destinasi yang relevan dari total dataset yang tersedia berdasarkan preferensi pengguna. Jika nilai *Recall* tinggi, berarti sistem sangat efektif dalam cakupan (*coverage*) dan tidak melewatkan destinasi-destinasi potensial (*hidden gems*) yang seharusnya disukai oleh pengguna tersebut. Metrik ini menjadi penyeimbang agar sistem tidak hanya memberikan hasil yang terbatas, melainkan mampu mengeksplorasi seluruh koleksi data destinasi wisata secara luas (Esmeli dkk., 2023).

2.10.3 *F1-Score*

F1-Score merupakan rata-rata harmonik dari *Precision* dan *Recall*. Metrik ini digunakan untuk mendapatkan nilai tunggal yang menyeimbangkan antara ketepatan dan kelengkapan hasil rekomendasi (Khumaidi dkk., 2025). Nilai *F1-Score* yang tinggi hanya bisa dicapai jika nilai *Precision* dan *Recall* keduanya tinggi secara bersamaan. Metrik ini menjadi indikator performa utama karena mampu meringkas kualitas retrieval sistem secara objektif dalam satu nilai skalar (Esmeli dkk., 2023). Rumusnya adalah (Khumaidi dkk., 2025):

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (10)$$

Keterangan:

- Precision@K*: Nilai ketepatan sistem yang dihitung pada subbab

2.10.1.

- b. *Recall@K*: Nilai kelengkapan sistem yang dihitung pada subbab 2.10.2.

2.11 Penelitian Terdahulu

Penelitian terdahulu menjadi landasan untuk memposisikan penelitian ini di antara riset-riset yang sudah ada. Fokus utama perbandingan terletak pada penggunaan metode *recommender system*, pengolahan teks semantik, dan jenis *feedback* yang digunakan. Berikut adalah ringkasan penelitian terdahulu yang relevan:

Tabel 2. 1 Penelitian Terdahulu

Penelitian terdahulu	Dataset	Metode	Permasalahan	Evaluasi
(Channarong dkk., 2022)	<i>MovieLens-1M</i> , <i>Yelp</i> , dan <i>Goodreads</i> .	<i>HybridBERT4Rec</i> (Integrasi BERT dan CF).	Metode rekomendasi tradisional statis gagal menangkap perubahan minat dinamis, serta mengabaikan interaksi urutan pengguna lain.	<i>Hit Ratio</i> (HR) dan <i>NDCG</i> .
Faurina & Sitanggang (2023)	75 destinasi wisata di Bali dan 3000 histori <i>rating</i> dari 100 pengguna.	<i>Content-Based Filtering</i> (TF-IDF & <i>Cosine Similarity</i>) + <i>Collaborative Filtering</i> .	Wisatawan kesulitan menemukan destinasi yang sesuai di Bali karena terlalu banyaknya informasi pariwisata yang tersedia (<i>Information Overload</i>).	<i>RMSE</i> (0.2427) dan <i>Loss</i> (0.0589).
Oyadila dkk. (2025)	Data destinasi wisata di Kabupaten Aceh Tengah.	<i>Content-Based Filtering</i> dengan TF-IDF dan <i>Cosine Similarity</i> .	Minimnya personalisasi rekomendasi sehingga wisatawan kebingungan mencari wisata di Aceh Tengah.	<i>Top-N Recommendation Test</i> (Akurasi 88%).
Gunathilaka dkk. (2025)	Beragam dataset publik (<i>MovieLens</i> ,	<i>Systematic Literature Review</i> (SLR).	Sistem rekomendasi sering mengalami	<i>Accuracy</i> , <i>Precision</i> , <i>Recall</i> , <i>MAE</i> , <i>MSE</i> .

Penelitian terdahulu	Dataset	Metode	Permasalahan	Evaluasi
	<i>Amazon, Netflix, dll).</i>		masalah <i>data sparsity</i> (minim interaksi) dan algoritma kesulitan memahami kemiripan tanpa bantuan teks profil item.	
Wardani dkk. (2025)	<i>Quora Question Pairs</i> (QQP) berlabel duplikat dan non-duplikat.	<i>Evaluasi model all-MiniLM-L6-v2 versus BERT-Base-Uncased.</i>	Model bahasa besar (BERT) membutuhkan sumber daya komputasi yang sangat tinggi dan lambat jika dieksekusi pada <i>server</i> standar.	<i>F1-Score (Model MiniLM terbukti 5,3 kali lebih cepat).</i>

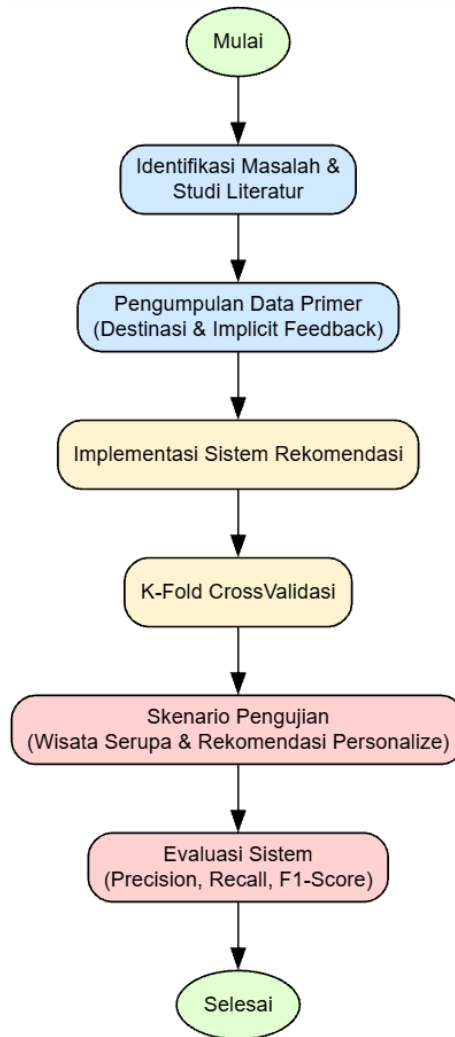
Berdasarkan tinjauan dari beberapa penelitian terdahulu di atas, terlihat adanya kesenjangan (*research gap*) pada sistem rekomendasi wisata yang ada saat ini. Oleh karena itu, penelitian yang diusulkan ini, yaitu sistem rekomendasi wisata Jember berbasis *implicit feedback* menggunakan metode *Hybrid Filtering*, hadir sebagai solusi untuk mengatasi sulitnya mendapatkan rekomendasi wisata yang relevan secara personal serta menanggulangi masalah kelebihan informasi (*information overload*) pada data destinasi regional di Kabupaten Jember.

Berbeda dengan pendekatan tradisional, penelitian ini menggunakan integrasi model pemrosesan bahasa alami all-MiniLM-L6-v2 untuk mengekstraksi fitur semantik dari destinasi wisata, serta mengkombinasikannya dengan *User-Item Interaction* yang murni berbasis *implicit feedback* berupa rekam jejak riwayat klik (*clickstream*). Sistem ini akan diimplementasikan dan diuji pada dataset yang mencakup 56 destinasi wisata di Kabupaten Jember beserta data simulasi interaksi pengguna. Untuk menjamin keandalan sistem, pengujian akan dievaluasi secara ketat menggunakan metode *K-Fold Cross Validation* dengan metrik pengukuran performa berupa *Precision@K*, *Recall@K*, dan *F1-Score*.

BAB 3 METODOLOGI PENELITIAN

3.1 Prosedur Penelitian

Penelitian ini menggunakan pendekatan eksperimental dengan metode eksperimen laboratorium dengan tujuan mengembangkan dan mengevaluasi sistem rekomendasi berbasis data *Implicit Feedback* menggunakan metode *Hybrid Filtering*, yang diintegrasikan dengan model *Transformer* all-MiniLM-L6-v2 untuk ekstraksi fitur semantiknya. Prosedur penelitian disusun secara terstruktur untuk memastikan integrasi antara data *Implicit Feedback* dan *vector database* berjalan optimal. Alur tahapan penelitian secara keseluruhan digambarkan dalam bentuk diagram alir pada Gambar 3.1 sebagai berikut:



Gambar 3. 1 Diagram Alir Prosedur Penelitian

3.1.1 Identifikasi Masalah & Studi Literatur

Tahap awal dimulai dengan menganalisis fenomena *information overload* pada informasi pariwisata di Kabupaten Jember. Pada tahap ini, dipelajari pula konsep metode *Hybrid Filtering* dan *Semantic Similarity* sebagai solusi untuk mengatasi masalah *data sparsity* yang sering terjadi pada sistem berbasis *rating* eksplisit.

3.1.2 Tahap Pengumpulan Data

Data dalam penelitian ini seluruhnya dikategorikan sebagai data primer, yaitu yang dikumpulkan secara mandiri oleh peneliti. Data tersebut dibagi menjadi dua jenis berdasarkan sifat dan sumber perolehannya, yaitu:

1. Data Destinasi Wisata

Data destinasi wisata dalam penelitian ini adalah dataset statis yang berisi informasi detail mengenai destinasi pariwisata di Kabupaten Jember dengan total wisata yang ada didalam (*destinasi_processed.csv*) berjumlah 56 destinasi wisata. Pengumpulan data dilakukan melalui metode Observasi Digital dan Kurasi Data Manual.

- Informasi dikumpulkan dari portal resmi Dinas Pariwisata dan Kebudayaan Kabupaten Jember serta situs informasi pariwisata terkait yang kredibel.
- Data diekstraksi secara manual yang meliputi nama wisata, kategori, alamat, harga tiket, gambar, kota, dan deskripsi naratif, kemudian disusun ke dalam format tabulasi (*destinasi_processed.csv*) untuk menjamin kualitas dan kebersihan data yang akan diolah oleh model AI seperti yang ditampilkan di gambar 3.2.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	nama	alamat	deskripsi	gambar	foto	jarak																
2	Pantai Rembang	Desa Rembang, Kecamatan Rembang, Kabupaten Jember	Pantai Rembang adalah destinasi wisata yang menawarkan udara sejuk, panorama alam nan mempesona, serta																			
3	Pantai Witu	Pantai Witu, Kecamatan Witu, Kabupaten Jember	Pantai Witu terkenal karena hata karang panjang yang menyempai ulat (jatu = batu, ulu = ulat) yang menjalar ke laut, menjadi ikon pantai ini. Ter																			
4	Pantai Kopo & Kalao	Kepulauan Kopo, Kecamatan Kopo, Kabupaten Jember	Pantai Kopo & Kalao adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
5	Kabin Cagar Budaya	Kabin Cagar Budaya, Kecamatan Jember, Kabupaten Jember	Kabin Cagar Budaya adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
6	Makam Habib Saleh	Makam Habib Saleh, Kecamatan Jember, Kabupaten Jember	Makam Habib Saleh adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
7	Desa Wisata Sidomulyo	Desa Wisata Sidomulyo, Kecamatan Sidomulyo, Kabupaten Jember	Desa Wisata Sidomulyo adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
8	Pemandian Keron Ageng	Pemandian Keron Ageng, Kecamatan Keron, Kabupaten Jember	Pemandian Keron Ageng adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
9	Pantai Pancer	Pantai Pancer, Kecamatan Pancer, Kabupaten Jember	Pantai Pancer adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
10	Jember Mini Zoo	Jember Mini Zoo, Kecamatan Jember, Kabupaten Jember	Jember Mini Zoo adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
11	Kimba Camp	Kimba Camp, Kecamatan Kimba, Kabupaten Jember	Kimba Camp adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
12	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
13	Pantai Pancer	Pantai Pancer, Kecamatan Pancer, Kabupaten Jember	Pantai Pancer adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
14	Air Terjun Tancak	Air Terjun Tancak, Kecamatan Tancak, Kabupaten Jember	Air Terjun Tancak adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
15	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
16	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
17	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
18	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
19	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
20	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
21	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
22	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
23	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
24	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
25	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
26	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
27	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
28	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			
29	Pantai Payung	Pantai Payung, Kecamatan Payung, Kabupaten Jember	Pantai Payung adalah destinasi wisata yang menawarkan pemandangan alam yang indah, dengan pasir putih yang lembut dan air yang jernih. Pantai ini																			

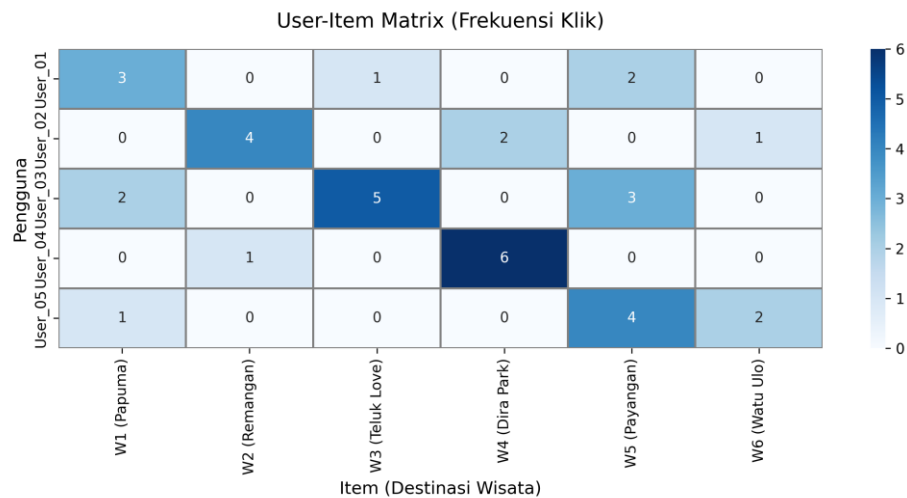
Gambar 3. 2 Data Destinasi Wisata Jember (*destinasi_Processed.csv*)

- c) Dataset ini berfungsi sebagai pre-collected data set yang memungkinkan sistem melakukan simulasi rekomendasi awal tanpa interaksi pengguna langsung (offline experiment).

2. Data *Implicit Feedback*

Data *Implicit Feedback* dikumpulkan secara dinamis melalui interaksi langsung pengguna dengan aplikasi JemberTrip. Data ini berperan sebagai dasar utama dalam metode *Memory-Based Collaborative Filtering*.

- Sistem tidak menggunakan *rating* eksplisit (seperti bintang 1-5), melainkan merekam perilaku pengguna berupa riwayat klik (*clickstream*).
- Sistem secara otomatis merekam perilaku klik pengguna terhadap detail destinasi wisata. Data tersebut disimpan dalam basis data relasional sebagai *Implicit feedback* yang nantinya digunakan untuk membangun profil preferensi pengguna seperti yang ditampilkan pada gambar 3.3.



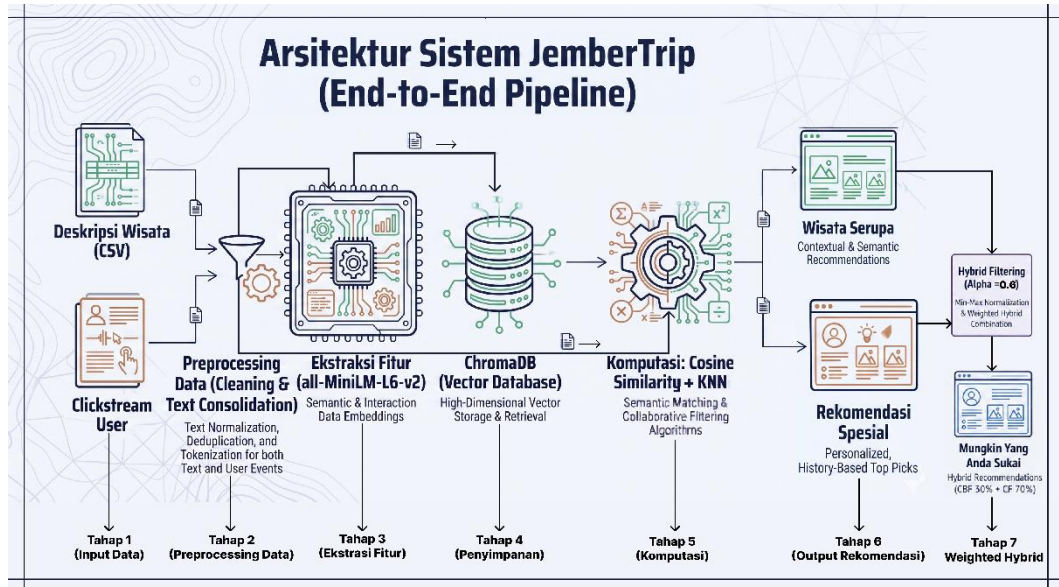
Gambar 3. 3 Simulasi Data *Implicit Feedback*

- c) Jejak aktivitas ini digunakan untuk membangun profil preferensi pengguna secara implisit, di mana frekuensi klik pada kategori tertentu dianggap sebagai sinyal minat yang kuat terhadap destinasi serupa.

3.1.3 Implementasi Sistem Rekomendasi

Pada bagian ini, dirancang alur pemrosesan data (*pipeline*) yang berfungsi mentransformasikan data tekstual destinasi wisata Jember beserta data riwayat interaksi pengguna (*Implicit Feedback*) menjadi representasi vektor numerik. Arsitektur *pipeline* ini merupakan komponen inti yang memungkinkan sistem

menjalankan fungsi algoritma rekomendasi berupa Wisata Serupa dan Rekomendasi *Personalize* secara akurat. Arsitektur AI *pipeline* ini ditunjukkan pada Gambar 3.4 berikut:



Gambar 3. 4 Arsitektur AI *Pipeline* JemberTrip

Detail fungsional dari setiap tahapan dalam *pipeline* tersebut adalah sebagai berikut:

1. Tahap Input Data

Pada tahap paling awal, sistem dirancang untuk menerima dua sumber data utama yang akan diproses secara bersamaan. Sumber data pertama adalah Deskripsi Wisata (CSV), yang merupakan data statis berisi atribut lengkap destinasi wisata di Jember. Sumber data kedua adalah *Clickstream* User (Basis Data Relasional), yang merepresentasikan aliran data dinamis berupa riwayat interaksi atau jejak klik pengguna terhadap destinasi wisata di dalam aplikasi yang ditarik secara *real-time* dari penyimpanan SQLite (untuk lingkungan pengembangan lokal) atau PostgreSQL (untuk peladen produksi/hosting). Kedua sumber data ini merupakan bahan baku utama yang sangat krusial sebelum sistem dapat mengenali pola preferensi dan kecocokan antar destinasi.

2. Tahap Preprocessing Data

Sebelum data diolah oleh model *Transformer* (All-MiniLM-L6-v2) untuk ekstraksi fitur, sistem melakukan tahap persiapan dan pembersihan

(*preprocessing*) terhadap dua sumber data utama. Tahapan ini sangat penting untuk memastikan model kecerdasan buatan menerima konteks teks yang bersih dan utuh. Proses *preprocessing* ini terbagi menjadi dua, yaitu:

1) Preprocessing Data Destinasi Wisata (Data Statis)

Proses ini berfokus pada persiapan data dari dokumen CSV yang memuat atribut lengkap mengenai berbagai destinasi wisata di Jember. Tahapan perancangan yang dilakukan pada aliran data ini meliputi:

Tabel 3. 1 Tahap Preprocessing Data Destinasi Wisata

Tahapan Perancangan	Deskripsi
<i>Data Cleaning</i>	Memindai seluruh baris data destinasi secara otomatis guna mendeteksi adanya nilai atribut yang kosong atau tidak lengkap. Nilai-nilai kosong tersebut akan dibersihkan dan diganti dengan label teks standar (misalnya Tidak ada data) agar proses ekstraksi representasi vektor tidak mengalami ambiguitas atau kegagalan komputasi saat membaca dataset.
<i>Data Consolidation</i>	Menggabungkan kolom nama wisata, kategori, alamat, dan deskripsi detail menjadi satu kesatuan blok teks (<i>document content</i>). Penggabungan ini penting agar model AI memiliki konteks informasi yang lengkap mengenai suatu destinasi dalam satu representasi vektor.

2) Preprocessing Data *Clickstream* User (*Implicit Feedback*)

Proses ini secara khusus menangani aliran data riwayat interaksi (*clickstream*) yang terekam secara *real-time* ketika pengguna menggunakan aplikasi. Data dinamis ini harus diproses terlebih dahulu sebelum dapat digunakan sebagai kueri untuk mencari rekomendasi personal. Tahapan yang dilakukan meliputi:

Tabel 3. 2 Tahap Preprocessing Data *Clickstream* User

Tahapan Perancangan		Deskripsi
<i>Frequency</i> (Penghitungan Interaksi)	<i>Counting</i> Frekuensi	Sistem mengumpulkan dan menarik (<i>retrieve</i>) riwayat <i>implicit feedback</i> dengan cara menghitung total akumulasi klik yang dilakukan oleh seluruh pengguna terhadap masing-masing destinasi wisata. Sistem mengambil seluruh data secara penuh untuk mendapatkan sebaran data perilaku (<i>behavior</i>) pengguna yang paling maksimal.
<i>Text Aggregation</i> (Agregasi Kueri Teks untuk CBF)		Sistem mengekstrak nama-nama destinasi wisata dari seluruh riwayat klik tersebut, kemudian

Tahapan Perancangan	Deskripsi
	menggabungkannya menjadi satu kalimat kueri teks yang terpadu (<i>query_text</i>). Teks agregasi inilah yang dikirim ke tahap ekstraksi fitur (SBERT) untuk dicocokkan kemiripan kontennya di metode CBF.
<i>User-Item Matrix Construction</i> (Pembentukan Matriks)	Angka frekuensi interaksi (jumlah klik) yang telah dihitung kemudian ditransformasikan secara <i>pivoting</i> ke dalam sebuah tabel matriks berdimensi dua. Baris (<i>row</i>) pada matriks mewakili entitas Pengguna (<i>Users</i>), sedangkan kolom (<i>column</i>) mewakili entitas Destinasi Wisata (<i>Items</i>). Matriks angka inilah yang menjadi pondasi utama sebelum dilakukan komputasi <i>Cosine Similarity</i> .

3. Tahap Ekstraksi Fitur (Model all-MiniLM-L6-v2)

Berdasarkan arsitektur *End-to-End Pipeline* yang telah diilustrasikan pada Gambar 3.4, data teks yang telah dikonsolidasi pada tahap *preprocessing* selanjutnya dialirkan ke dalam modul Ekstraksi Fitur. Pada perancangan sistem JemberTrip ini, diimplementasikan model *Transformer* all-MiniLM-L6-v2 sebagai mesin utama untuk melakukan *Sentence embedding*.

Dalam implementasinya di dalam sistem, model ini tidak dilatih ulang secara manual, melainkan langsung difungsikan sebagai *Encoder* statis. Tugas utamanya adalah mentransformasikan teks bahasa manusia menjadi representasi vektor numerik (titik koordinat dalam ruang 384 dimensi) melalui mekanisme komputasi self-attention seperti yang tertuang pada Persamaan (3) di Bab 2, agar komputer dapat menghitung kemiripan maknanya secara matematis. Secara fungsional, proses ekstraksi fitur pada sistem ini dirancang untuk memproses dua aliran teks yang menghasilkan dua jenis keluaran vektor khusus untuk *Content-Based Filtering*, yaitu:

1) *Semantic Data Embeddings* (Ekstraksi Data Destinasi Wisata)

Sistem akan mengekstrak blok teks dari setiap destinasi wisata (gabungan nama, kategori, alamat, dan deskripsi) yang berasal dari file CSV. Hasil ekstraksi ini berupa representasi vektor yang merepresentasikan identitas semantik atau karakteristik konten dari masing-masing destinasi wisata tersebut.

2) *User Query Embeddings* (Ekstraksi Data *Clickstream* User)

Pada saat yang bersamaan, sistem juga mengekstrak teks kueri yang dihasilkan dari agregasi seluruh riwayat klik (*clickstream*) pengguna. Hasil ekstraksi ini akan menjadi sebuah vektor tunggal yang merepresentasikan profil minat pengguna secara *real-time*. Vektor inilah yang akan dijadikan kueri pencarian utama untuk dicocokkan dengan vektor destinasi wisata pada metode *Content-Based Filtering*.

4. Tahap Penyimpanan (*ChromaDB Vector database*)

Berdasarkan rancangan *pipeline* pada Gambar 3.4, vektor numerik yang telah dihasilkan oleh model *Transformer* selanjutnya ditampung pada basis data vektor, yaitu ChromaDB. Pada implementasi sistem JemberTrip, ChromaDB tidak sekadar menyimpan kumpulan angka secara acak, melainkan difungsikan sebagai *High-Dimensional Vector Storage & Retrieval* melalui mekanisme penyimpanan objek dokumen yang terstruktur.

Dalam proses implementasinya, sistem mengatur format penyimpanan untuk setiap objek destinasi wisata ke dalam dua elemen utama penyusun basis data vektor, yaitu:

1) Penyimpanan Ruang Vektor (*Document Content*)

Sistem memasukkan hasil ekstraksi teks gabungan destinasi wisata (sebagai *Semantic Data Embeddings*) ke dalam ruang indeks ChromaDB. Indeksasi ruang vektor ini memastikan agar sistem nantinya dapat mengukur kedekatan jarak antarvektor secara efisien pada saat pencarian berlangsung.

2) Penyematan Atribut Pelengkap (*Metadata*)

Bersamaan dengan penyimpanan titik koordinat vektor, sistem dirancang untuk mengikatkan data pelengkap (*metadata*) pada setiap titik vektor destinasi tersebut. Atribut yang disematkan secara spesifik meliputi ID destinasi, nama wisata, kategori, alamat lengkap, dan direktori letak gambar.

5. Tahap Komputasi (*Cosine Similarity* + KNN)

Tahap komputasi merupakan mesin penggerak utama dari algoritma rekomendasi JemberTrip, di mana terjadi pertemuan komputasional antara representasi vektor yang telah tersimpan di dalam basis data dengan vektor kueri yang masuk secara seketika (*real-time*). Pada tahapan ini, sistem dirancang untuk mengimplementasikan *Semantic Matching* guna menemukan tingkat kecocokan antar data tanpa bergantung pada kesamaan kata kunci secara harfiah. Sebelum komputasi dilakukan, sistem menerapkan aturan penyaringan awal dengan hanya memanggil data yang memiliki label identifikasi *tourism*, sehingga mesin komputasi terhindar dari pemrosesan data pengetahuan (*knowledge base*) yang tidak relevan.

Secara arsitektur, mesin komputasi dirancang untuk menangani dua jenis pemrosesan berdasarkan skenario rekomendasi yang dijalankan:

Tabel 3. 3 Jenis Pemrosesan Berdasarkan Skenario Rekomendasi

Jenis Pemrosesan			Deskripsi
Komputasi Wisata Serupa (<i>Item-to-Item</i>)			Vektor kueri pengguna (<i>User Query Embeddings</i>) yang baru saja diekstraksi dialirkan ke mesin komputasi untuk dihadapkan dengan seluruh koleksi vektor destinasi wisata di dalam <i>ChromaDB</i> . Sistem mengeksekusi perhitungan <i>Cosine Similarity</i> untuk mencari destinasi mana saja yang makna semantik kontennya paling mirip dengan riwayat klik pengguna.
Komputasi Rekomendasi <i>Personalize (User-to-Item)</i>			Berbeda dengan CBF, mesin ini tidak melibatkan vektor ataupun <i>ChromaDB</i> . Sistem mengambil <i>User-Item Matrix</i> (matriks frekuensi klik), kemudian mengeksekusi perhitungan <i>Cosine Similarity</i> antar baris pengguna untuk mencari tahu seberapa mirip selera pengguna saat ini dengan selera pengguna lainnya di dalam basis data.

Dalam melakukan pencocokan pada kedua skenario di atas, mesin komputasi menggunakan kalkulasi metrik *Cosine Similarity* seperti yang telah didefinisikan pada Persamaan (5) di Bab 2. Algoritma ini bekerja dengan menghitung nilai kosinus dari sudut yang terbentuk antara query vector dengan vektor-vektor target di dalam ruang dimensi. Semakin sempit sudut yang terbentuk di antara kedua vektor, maka skor *Cosine Similarity* akan semakin tinggi (mendekati angka 1), yang secara matematis

menandakan bahwa kedua data tersebut memiliki konteks, karakteristik, dan makna semantik yang sangat mirip.

Setelah skor kemiripan berhasil dikalkulasi, sistem mengaplikasikan algoritma K-Nearest Neighbors (KNN) sebagai fungsi penyaringan. Algoritma KNN bertugas menyortir hasil kalkulasi secara menurun (descending) dan mengekstraksi sejumlah nearest neighbors. Berdasarkan rancangan evaluasi sistem, parameter KNN dikonfigurasi secara presisi untuk menyaring 10 nearest neighbors (Top-10) pada eksekusi pencarian Wisata Serupa (CBF), dan menyaring 30 nearest neighbors (K=30) pada eksekusi Rekomendasi Personalize (CF). Khusus untuk metode CF, 30 nearest neighbors tersebut kemudian dihitung nilai prediksinya menggunakan algoritma *Weighted Sum* berdasarkan Persamaan (2) yang telah diuraikan pada Bab 2 sebelum akhirnya dikirim ke tahapan output."

6. Output Rekomendasi

Tahap ini merupakan proses hilir bagi algoritma tunggal (*single algorithm*) pada arsitektur *End-to-End Pipeline* JemberTrip. Pada tahap ini, hasil komputasi dari *Content-Based Filtering* (CBF) maupun *Collaborative Filtering* (CF) tidak serta-merta langsung disajikan ke layar pengguna. Sistem dirancang untuk melakukan satu lapisan validasi akhir (*Exclusion Filtering*) guna memastikan bahwa rekomendasi yang diberikan benar-benar logis dan memberikan pengalaman penemuan baru (*novelty*).

Sistem akan mengeksekusi aturan validasi eksklusi yang berbeda menyesuaikan dengan skenario fitur yang sedang berjalan:

Tabel 3. 4 Skenario Validasi Eksklusi Berdasarkan
Fitur Rekomendasi

Skenario Fitur	Aturan Eksklusi (Penyaringan)	Tujuan Eksklusi	Batasan Keluaran (Output)
Wisata Serupa	Mengecualikan/menghapus ID destinasi acuan yang sedang dibuka oleh pengguna dari daftar hasil komputasi.	Mencegah anomali di mana sistem merekomendasikan tempat wisata yang sama persis dengan yang sedang dilihat pengguna.	Top-3 (3 rekomendasi teratas)

Skenario Fitur	Aturan Eksklusi (Penyaringan)	Tujuan Eksklusi	Batasan Keluaran (Output)
Rekomendasi Personalize	Mengeliminasi destinasi-destinasi wisata yang sebelumnya sudah pernah diklik atau dikunjungi oleh pengguna.	Memberikan variasi tempat wisata baru (<i>novelty</i>) agar tidak merekomendasikan tempat yang sudah diketahui pengguna.	Top-6 (6 rekomendasi teratas)

Setelah melewati tahap penyaringan ini, hasil komputasi dari algoritma tunggal tersebut langsung didistribusikan ke lapisan antarmuka (Front-End) menjadi dua panel utama, yaitu panel Wisata Serupa (menggunakan hasil CBF) dan panel Rekomendasi Spesial (menggunakan hasil CF).

7. *Weighted Hybrid*

Sebagai tahapan pamungkas (*final stage*) komputasi dari arsitektur *End-to-End Pipeline* JemberTrip, sistem mengeksekusi algoritma *Hybrid Filtering*. Tahapan ini bertujuan untuk menutupi kelemahan *Collaborative Filtering* (seperti masalah pengguna baru yang minim interaksi) dengan memanfaatkan kelebihan *Content-Based Filtering*. Proses penggabungan ini dilakukan melalui dua mekanisme:

Tabel 3. 5 *Hybrid Filtering*

Tahapan Komputasi	Deskripsi Mekanisme
<i>Min-Max Normalization</i>	Sistem mengonversi skor kemiripan mentah (<i>raw score</i>) yang dihasilkan oleh <i>Content-Based Filtering</i> dan <i>Collaborative Filtering</i> menggunakan perhitungan normalisasi <i>Min-Max</i> pada persamaan (1), sehingga seluruh nilai berubah menjadi skala rasio desimal dengan rentang yang seragam antara angka 0 hingga 1.
<i>Weighted Hybrid Combination</i>	Skor yang telah ternormalisasi dijumlahkan menggunakan parameter bobot (<i>Alpha</i>) berdasarkan Persamaan (2) yang merujuk pada landasan teori di Bab 2. Berdasarkan pengujian evaluasi skenario, sistem ini dikonfigurasi menggunakan bobot <i>Alpha</i> = 0.6, yang berarti skor prediksi akhir setiap destinasi dihitung dengan mengambil porsi dominan sebesar 60% dari tebakan algoritma <i>Collaborative Filtering</i> , dan porsi pendukung sebesar 40% dari tebakan algoritma <i>Content-Based Filtering</i> .

Setelah melewati tahapan komputasi *Hybrid* tersebut, sistem akan membongkar atribut *metadata* yang melekat pada destinasi-destinasi terpilih (seperti nama wisata, kategori, alamat, dan tautan gambar), lalu mendistribusikannya ke lapisan antarmuka (*Front-End*) secara seketika (*real-time*). Berdasarkan rancangan arsitektur, keluaran akhir ini diimplementasikan ke dalam tiga panel antarmuka utama di dalam aplikasi:

Tabel 3. 6 Implementasi Panel Antarmuka Keluaran Sistem

Panel Antarmuka	Bentuk / Lokasi Tampilan	Deskripsi dan Basis Rekomendasi
Wisata Serupa	Deretan kartu destinasi pada halaman detail wisata	Menampilkan destinasi wisata alternatif yang secara konteks, karakteristik, dan suasana semantik memiliki kemiripan dengan objek wisata yang saat itu sedang dipelajari oleh pengguna.
Rekomendasi <i>Personalize</i>	Halaman beranda (dashboard) pengguna	Menampilkan daftar rekomendasi wisata eksklusif yang dirancang khusus untuk masing-masing pengguna, dihasilkan dari kalkulasi rekam jejak riwayat pengguna lain (<i>Collaborative Filtering</i>).
Mungkin Yang Anda Sukai	Deretan kartu destinasi pada halaman beranda (<i>dashboard</i>) pengguna di bawah panel Rekomendasi <i>Personalize</i>	Menampilkan rekomendasi hasil penggabungan (<i>Hybrid Filtering</i>) dari tebakan <i>Content-Based Filtering</i> dan <i>Collaborative Filtering</i> menggunakan bobot <i>Alpha</i> 0.6, memberikan hasil yang paling akurat dengan meminimalisir kelemahan dari masing-masing algoritma tunggal.

Dengan disajikannya ketiga antarmuka output ini, maka berakhirilah satu siklus penuh pemrosesan dari arsitektur *End-to-End Pipeline* JemberTrip; dimulai dari penyerapan data mentah berupa teks statis dan riwayat klik dinamis, hingga bertransformasi menjadi sistem rekomendasi *Hybrid* cerdas yang adaptif, akurat, dan sangat personal bagi setiap pengguna.

3.1.4 *K-Fold Cross Validation*

Mengingat sistem JemberTrip mengandalkan pendekatan *Hybrid Filtering* yang menggabungkan kemiripan semantik konten (*Content-Based Filtering*) dan profil interaksi pengguna (*Collaborative Filtering*), maka diperlukan metode

pengujian yang mampu mengukur stabilitas performa algoritma secara menyeluruh. Dalam penelitian ini, validasi dilakukan untuk menguji sejauh mana mesin algoritma rekomendasi *Hybrid* ini mampu mempertahankan konsistensi akurasi ketika dihadapkan pada variasi penyebaran data riwayat interaksi klik yang berbeda-beda. Oleh karena itu, prosedur validasi dijalankan secara sistematis menggunakan teknik 10-Fold Cross Validation dengan rincian sebagai berikut:

- a) Dataset simulasi riwayat interaksi (*clickstream*) pengguna dibagi secara acak ke dalam 10 bagian (*folds*) yang memiliki ukuran sama besar.
- b) Dalam setiap putaran pengujian, satu bagian (*fold*) akan berfungsi sebagai data uji (*test set*), sementara 9 bagian lainnya ($K - 1$) bertindak sebagai data pendukung atau acuan bagi sistem.
- c) Prosedur ini diulang sebanyak 10 kali secara bergantian hingga setiap data di dalam dataset pernah digunakan sebagai data uji tepat satu kali.
- d) Hasil evaluasi yang dilaporkan dalam penelitian ini merupakan nilai rata-rata (*mean*) dari seluruh putaran validasi tersebut guna memberikan gambaran performa sistem yang lebih stabil.

3.1.5 Skenario Pengujian

Pengujian dilakukan dengan metode eksperimen kuantitatif guna memvalidasi efektivitas sistem JemberTrip. Selaras dengan rumusan masalah yang ditetapkan, pengujian ini bertujuan untuk mengukur dan membandingkan tingkat akurasi dari tiga metode rekomendasi utama, yaitu *Content-Based Filtering*, *Memory-Based Collaborative Filtering*, dan *Hybrid Filtering*. Guna mendapatkan gambaran performa yang komprehensif, pengujian dibagi menjadi tiga skenario utama yang dievaluasi menggunakan teknik 10-Fold Cross Validation, sebagai berikut:

1. Skenario Pengujian Wisata Serupa (*Item-to-Item*)

Skenario ini bertujuan mengukur kemampuan sistem dalam menemukan destinasi yang memiliki kemiripan semantik dengan destinasi yang sedang dibuka oleh pengguna. Prosedur pengujian dilakukan sebagai berikut:

- a) Seluruh destinasi dalam dataset (56 item) dibagi menjadi 10 *fold* untuk divalidasi secara bergantian.
- b) Pada setiap iterasi pengujian, setiap destinasi yang berada di dalam test

fold akan bertindak sebagai acuan.

- c) Vektor deskripsi dari destinasi acuan (yang telah diekstraksi oleh model all-MiniLM-L6-v2) dihitung nilai *Cosine Similarity*-nya terhadap seluruh vektor destinasi di dalam *training fold*.
- d) Sistem menghasilkan daftar Top-6 destinasi wisata serupa untuk setiap acuan.

Relevansi ditentukan berdasarkan kategori destinasi acuan. Skenarionya adalah jika destinasi acuan yang sedang dibuka pengguna berkategori wisata pantai, maka hasil rekomendasi sistem divalidasi sebagai relevan jika destinasi tersebut juga memiliki label kategori wisata pantai atau karakteristik semantik yang sebanding dalam dataset.

Setiap Item dalam daftar *Top-6 Recommendation* akan diberikan bobot biner sebagai berikut:

- a) Label 1: Jika Kategori item sesuai dengan *Ground Truth* (relevan)
- b) Label 0: Jika kategori item tidak sesuai dengan *Ground Truth* (Tidak Relevan)

2. Skenario Pengujian Rekomendasi *Personalize (User-to-Item)*

Skenario ini menguji efektivitas sistem dalam merekomendasikan wisata berdasarkan kemiripan pola interaksi antar pengguna. Prosedur pengujian dilakukan sebagai berikut:

- a) Peneliti menyiapkan 100 profil pengguna simulasi yang dibagi ke dalam 10 *fold* (10 profil per *fold*). Pada setiap iterasi, 9 *fold* digunakan sebagai data histori latih (Training Set), dan 1 *fold* digunakan sebagai data uji (*Test Set*).
- b) Sistem membangun *User-Item Matrix* (matriks frekuensi klik) berdasarkan data *Training Set*. Sistem kemudian menghitung kemiripan selera antar baris pengguna menggunakan metrik *Cosine Similarity* dan menyaring *nearest neighbors* menggunakan parameter optimal $K=30$.
- c) Sistem menghitung probabilitas prediksi menggunakan rumus *Weighted Sum*, lalu menyortir hasil dan mengeluarkan daftar rekomendasi Top-6.

3. Skenario Pengujian *Hybrid Filtering (Weighted Hybrid)*

Skenario ini merupakan puncak pengujian untuk membuktikan bahwa metode gabungan memiliki performa dan akurasi yang lebih baik dibandingkan masing-masing algoritma tunggal. Prosedur pengujian dilakukan sebagai berikut:

- a) Sistem mengambil hasil perhitungan kemiripan mentah dari skenario *Content-Based Filtering* dan *Collaborative Filtering*
- b) Kedua nilai tersebut disetarakan menggunakan algoritma *Min-Max Normalization*.
- c) Nilai yang telah ternormalisasi digabungkan menggunakan rumus *Weighted Hybrid Combination* dengan parameter pembobotan $\alpha = 0.6$ (60% menggunakan tebakan *Collaborative Filtering*, dan 40% menggunakan tebakan *Content-Based Filtering*)
- d) Sistem menyortir hasil komputasi dan mengeluarkan Top-6 rekomendasi akhir.

Khusus pada skenario *Collaborative Filtering* dan *Hybrid Filtering*, sistem menentukan relevansi (*Ground Truth*) tidak dinilai berdasarkan kesamaan label kategori secara manual, melainkan diukur secara sistematis dengan mencocokkan hasil rekomendasi sistem (Top-6) terhadap data *Test Set* (yakni data destinasi yang secara faktual benar-benar diklik atau dikunjungi oleh pengguna pada simulasi tersebut).

Setiap Item dalam daftar *Top-6 recommendation* akan diberikan bobot biner sebagai berikut:

- a) Label 1: Jika destinasi yang direkomendasikan oleh sistem ternyata ada di dalam catatan riwayat *Test Set* pengguna (relevan).
- b) Label 0: Jika destinasi yang direkomendasikan oleh sistem ternyata tidak ada di dalam catatan riwayat *Test Set* pengguna (tidak relevan).

Hasil biner (relevan/tidak relevan) dari ketiga skenario tersebut kemudian diproses menggunakan kalkulasi metrik evaluasi matematis (*Precision*, *Recall*, dan *F1-Score*) untuk ditarik kesimpulan akhirnya.

3.1.6 Evaluasi Sistem

Setelah dilakukan *10-Fold Cross Validation* pada subbab 3.1.4 dan skenario pengujian pada subbab 3.1.5 dijalankan dan standar relevansi (*Ground Truth*) ditetapkan pada subbab 3.1.5, peneliti melakukan evaluasi performa sistem JemberTrip melalui tahapan sistematis berikut :

1. Pengukuran Ketepatan (*Precision@K*)

Precision@K ($P@K$) digunakan untuk mengukur seberapa banyak destinasi yang relevan dari total K item yang direkomendasikan. Perhitungan ini dilakukan pada setiap *fold* dalam prosedur *10-Fold Cross Validation* berdasarkan Persamaan (8) yang telah dipaparkan pada Bab 2.

- a) Peneliti mengambil daftar rekomendasi *Top-N* dengan nilai $k = 6$ untuk seluruh metode rekomendasi.
- b) Pada setiap *fold*, sistem membandingkan daftar 6 destinasi hasil rekomendasi terhadap *Ground Truth* (data destinasi yang benar-benar diklik/relevan bagi pengguna dalam *test set*).
- c) Menghitung nilai *Precision* untuk iterasi tersebut. Sebagai contoh, jika dalam satu *fold* terdapat 4 destinasi relevan dari total $k = 6$ yang ditampilkan, maka nilai $P@6$ untuk *fold* tersebut adalah 0,67.
- d) Nilai *Precision* final didapatkan dari rata-rata (*mean*) hasil perhitungan $P@6$ di seluruh 10 *fold* pengujian untuk menjamin objektivitas performa sistem secara keseluruhan.

2. Pengukuran Kelengkapan (*Recall@K*)

Recall@K ($R@K$) mengukur kemampuan sistem dalam menemukan kembali seluruh destinasi relevan yang tersedia di dalam dataset terhadap minat pengguna. Metrik ini dihitung sesuai dengan Persamaan (9) pada Bab 2 dengan mengikuti prosedur berikut:

- a) Sistem mengidentifikasi total ukuran *Ground Truth* sebagai penyebut (*denominator*). Pada skenario *Content-Based Filtering*, penyebutnya adalah total destinasi dengan kategori serupa di dalam dataset. Sedangkan pada skenario *Collaborative Filtering* dan *Hybrid Filtering*, penyebutnya adalah total jumlah klik faktual yang ada di dalam test set pengguna tersebut.

- b) Sama halnya dengan pengukuran presisi, sistem menerapkan batasan evaluasi pada daftar *Top-N* dengan nilai $k = 6$.
- c) Pada setiap iterasi dalam prosedur *10-Fold Cross Validation*, sistem menghitung berapa banyak destinasi relevan dari total populasi *Ground Truth* yang berhasil muncul di dalam daftar *Top-6* tersebut.
- d) Hasil *Recall* final didapatkan dari perhitungan rata-rata (*mean*) dari 10 kali putaran validasi untuk memastikan stabilitas cakupan (*coverage*) sistem.

3. Pengukuran Keseimbangan (*F1-Score*)

F1-Score digunakan sebagai metrik akhir untuk menyeimbangkan performa ketepatan (*Precision*) dan jangkauan (*Recall*) sistem. Perhitungan nilai ini merujuk pada Persamaan (10) di Bab 2. Dalam implementasinya pada JemberTrip, algoritma evaluasi *F1-Score* dilakukan secara sistematis melalui tahapan berikut:

- a) Mengumpulkan hasil *Precision@K* dan *Recall@K* yang telah dihitung pada setiap iterasi.
- b) Menghitung nilai rata-rata (*mean*) dari *Precision* dan *Recall* dari total 10 putaran pengujian untuk mendapatkan angka representatif yang stabil dan objektif.
- c) Memasukkan nilai rata-rata tersebut ke dalam rumus rata-rata harmonik sesuai Persamaan (9). Penggunaan rata-rata harmonik ini krusial untuk memastikan bahwa sistem tidak hanya unggul di satu sisi (misalnya hanya tepat menebak tapi cakupan variasinya sempit).
- d) Nilai akhir *F1-Score* digunakan untuk memvalidasi sejauh mana sistem rekomendasi JemberTrip secara keseluruhan (terutama metode *Hybrid*) mampu menjaga keseimbangan antara akurasi posisi atas (*high-ranking*) dan luasnya variasi destinasi yang relevan bagi pengguna.

3.2 Jadwal Tabel Penelitian

Jadwal penelitian disusun untuk memberikan gambaran manajemen waktu pelaksanaan Tugas Akhir dari tahap awal hingga penyelesaian. Berbeda dengan prosedur teknis penelitian, jadwal ini mencakup seluruh rangkaian administratif dan operasional, mulai dari penyusunan proposal, tahapan eksekusi algoritma

rekomendasi, penyusunan laporan akhir (skripsi), hingga tahap publikasi jurnal ilmiah.

Rincian jadwal pelaksanaan penelitian ini direncanakan berlangsung selama 3 (tiga) bulan, yang dimulai dari bulan Mei hingga Juli, dengan rincian kegiatan pada Tabel 3.7 berikut:

Tabel 3. 7 Jadwal Penelitian

NO	Tahapan	Bulan Ke-1				Bulan Ke-2				Bulan Ke-3			
		Minggu				Minggu				Minggu			
		1	2	3	4	1	2	3	4	1	2	3	4
1	Identifikasi masalah & Studi Literatur												
2	Pengumpulan Data (Dataset & Implicit Feedback)												
3	Implementasi Sistem Rekomendasi												
4	K-Fold Cross Validation												
5	Skenario Pengujian												
6	Evaluasi Sistem												

BAB 4 HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

Bagian ini menguraikan tahapan implementasi dari rancangan sistem rekomendasi wisata JemberTrip yang sebelumnya telah dimodelkan pada bab 3. Tahap implementasi bertujuan untuk mendokumentasikan proses transformasi dari desain dan alur algoritma menjadi sebuah perangkat lunak fisik yang fungsional dan siap diuji. Implementasi yang dibangun dalam penelitian ini mencakup penyiapan lingkungan peladen (*server*), pengelolaan basis data, hingga pembangunan antarmuka pengguna (*user interface*) yang memfasilitasi 3 fitur utama sistem, yaitu fitur pencarian Wisata Serupa menggunakan metode *Content-Based Filtering* dan fitur Rekomendasi Personalize menggunakan metode *Memory-Based Collaborative Filtering* yang digerakkan oleh data *implicit feedback*, serta fitur Mungkin Yang Anda Sukai menggunakan metode *Hybrid Filtering*.

4.1.1 Lingkungan Implementasi

Sistem rekomendasi wisata JemberTrip diimplementasikan pada dua lingkungan utama, yakni lingkungan perangkat keras (*hardware*) yang digunakan untuk tahap pengembangan dan pengujian komputasional lokal, serta lingkungan perangkat lunak (*software*) yang terbagi menjadi spesifikasi sisi peladen (*backend*) dan sisi antarmuka pengguna (*frontend*).

Rincian spesifikasi perangkat keras (*hardware*) yang digunakan sebagai mesin penggerak dalam penelitian ini disajikan pada Tabel 4.1 berikut:

Tabel 4. 1 Spesifikasi Perangkat Keras (*Hardware*)

Komponen	Spesifikasi yang Digunakan
Model Perangkat	Acer Predator Helios Neo 16
Sistem Operasi	Windows 11 <i>Home</i> 64-bit
Prosesor (CPU)	Intel Core i5- 14500HX
Kapasitas Memori (RAM)	16 GB DDR5
Media Penyimpanan	1TB NVMe Gen4 SSD

Berdasarkan Tabel 4.1, perangkat keras yang digunakan merupakan laptop berspesifikasi tinggi yang dilengkapi dengan prosesor Intel Core generasi ke-14 serta memori RAM 16 GB DDR5. Spesifikasi komputasi yang besar ini sangat krusial dan dibutuhkan guna memastikan proses validasi algoritma *Machine*

Learning, pembuatan vektor (*embedding*), serta kalkulasi *10-Fold Cross Validation* dapat berjalan dengan lancar tanpa kendala *memory-overflow*.

Selain perangkat keras, implementasi sistem ini juga ditopang oleh arsitektur perangkat lunak sisi peladen (*backend*). Lingkungan *backend* bertugas sebagai otak komputasi kecerdasan buatan, yang rinciannya disajikan pada Tabel 4.2 berikut:

Tabel 4. 2 Lingkungan Implementasi *Backend*

Komponen	Teknologi / Platform	Keterangan & Fungsi Utama
Bahasa Pemrograman	Python	Bahasa utama untuk memproses algoritma dan kecerdasan buatan.
Kerangka Kerja (<i>Framework</i>)	FastAPI	Kerangka kerja berkinerja tinggi berbasis <i>Asynchronous Server Gateway Interface</i> (ASGI) untuk pemrograman asinkron.
Pustaka (<i>Library</i>) Utama	Pandas, NumPy, Scikit-learn, Sentence-Transformers, Python-docx	Pustaka yang difungsikan untuk manipulasi matriks, ekstraksi teks, komputasi <i>Machine Learning</i> , dan pembentukan ruang vektor.
Basis Data (Lokal)	SQLite	Basis data relasional ringan (row-oriented).
Basis Data (Produksi)	PostgreSQL	Basis data relasional skala besar untuk merekam dan mengelola data dinamis <i>implicit feedback</i> (riwayat klik)
Platform <i>Hosting</i>	Railway	Layanan <i>cloud</i> untuk meng-hosting sistem

Merujuk pada Tabel 4.2, fondasi utama dari sistem kecerdasan buatan ini dibangun menggunakan bahasa pemrograman Python dan kerangka kerja FastAPI. Sistem ini secara khusus didukung oleh pustaka *Sentence-Transformers* untuk mengeksekusi model all-MiniLM-L6-v2, serta mengandalkan PostgreSQL untuk menangani rekam jejak aktivitas clickstream pengguna secara seketika (real-time). Seluruh ekosistem peladen ini kemudian di-hosting menggunakan layanan *cloud Railway*.

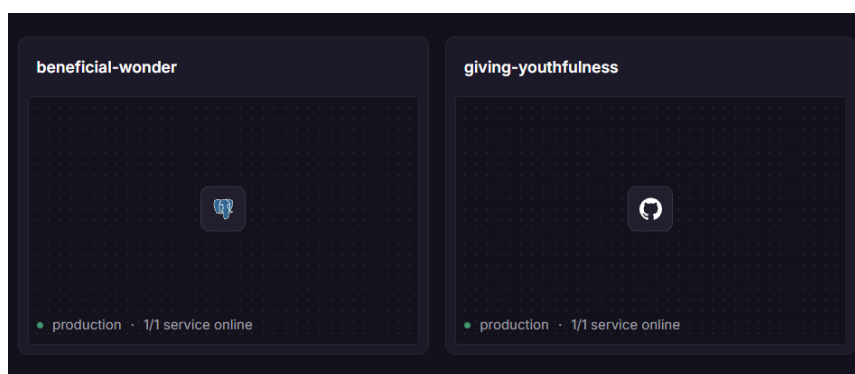
Selanjutnya, untuk memfasilitasi interaksi antara mesin rekomendasi dengan wisatawan, dibangunlah arsitektur perangkat lunak sisi antarmuka (*frontend*) yang rinciannya dijabarkan pada Tabel 4.3 berikut:

Tabel 4. 3 Lingkungan Implementasi *Frontend*

Komponen	Teknologi / Platform	Keterangan & Fungsi Utama
Platform <i>Hosting</i>	Vercel	Layanan komputasi awan (<i>cloud platform</i>) untuk menempatkan dan menyajikan antarmuka visual kepada publik secara interaktif.

Komponen	Teknologi / Platform	Keterangan & Fungsi Utama
Komunikasi Data	REST API dan AXIOS	alur komunikasi HTTP (<i>Application Programming Interface</i>) untuk mengirimkan data aktivitas klik pengguna ke <i>backend</i> secara <i>real-time</i> dan mengambil balasan (<i>response</i>) berupa daftar rekomendasi wisata.
Pustaka Utama	React JS	Pustaka berbasis JavaScript untuk membangun komponen antarmuka pengguna (<i>User Interface</i>) yang dinamis dan reaktif.
Build Tool	Vite	Alat pengembang (<i>build tool</i>) berkinerja tinggi untuk memproses kode secara asinkron dan mempercepat waktu <i>rendering</i> antarmuka.
Styling	Tailwind CSS	Kerangka kerja CSS berbasis <i>utility-first</i> untuk mendesain tata letak antarmuka yang modern, responsif, dan konsisten.

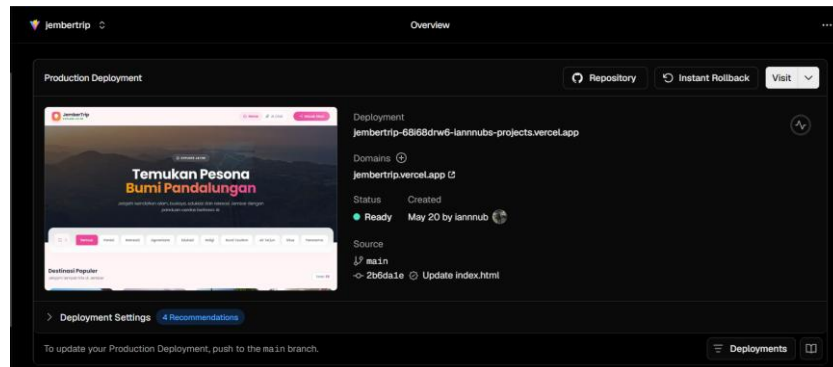
Berdasarkan Tabel 4.3, antarmuka pengguna dibangun secara modern menggunakan pustaka React JS dan kerangka kerja Tailwind CSS untuk menjamin responsivitas tata letak di berbagai ukuran layar perangkat. Secara visual, wujud utuh dari lingkungan implementasi pada sisi peladen (*backend*) dan antarmuka (*frontend*) tersebut dapat dilihat secara berturut-turut pada Gambar 4.1 dan Gambar 4.2 berikut. Kedua lingkungan (*software*) ini saling terhubung secara nirkabel melalui jalur komunikasi REST API dan AXIOS. Melalui jalur tersebut, *frontend* secara aktif mengirimkan rekaman aktivitas klik wisatawan, sementara sisi *backend* bertugas memprosesnya dan mengembalikan daftar rekomendasi wisata hasil komputasi model Machine Learning untuk ditampilkan di layar.



Gambar 4. 1 Sistem *Backend* dan Basis Data JemberTrip

Berdasarkan Gambar 4.1, dapat dilihat dasbor pemantauan dari platform cloud Railway yang bertindak sebagai pusat kendali (*command center*) sisi peladen. Pada lingkungan ini, seluruh logika kecerdasan buatan berbasis Python, kerangka kerja FastAPI, serta basis data PostgreSQL di-deploy secara terintegrasi agar dapat

memproses kueri aktivitas pengguna secara real-time dan terus-menerus (24/7) tanpa membebani komputasi perangkat lokal.



Gambar 4. 2 Sistem *Frontend* JemberTrip

Sementara itu, Gambar 4.2 menampilkan dasbor deployment dari platform Vercel yang menaungi sisi antarmuka (*frontend*) aplikasi JemberTrip. Lingkungan Vercel secara otomatis mem-build dan mendistribusikan kode program React JS menjadi halaman web statis yang sangat responsif. Melalui antarmuka publik yang di-hosting pada domain Vercel inilah, wisatawan dapat berinteraksi langsung dengan sistem rekomendasi tanpa perlu menyadari proses komputasi rumit yang sedang berjalan di latar belakang (*backend*).

4.1.2 Deskripsi Data Penelitian

Selain infrastruktur perangkat keras dan perangkat lunak, lingkungan implementasi dan pengujian sistem ini sangat bergantung pada ketersediaan dan kualitas data. Terdapat dua jenis dataset utama yang digunakan sebagai bahan baku untuk diproses oleh mesin kecerdasan buatan, yakni data statis destinasi wisata dan data dinamis interaksi pengguna (*implicit feedback*). Rincian karakteristik dan struktur dari kedua dataset tersebut disajikan pada Tabel 4.4 berikut:

Tabel 4. 4 Dataset JemberTrip

Jenis Dataset	Jumlah Data	Atribut / Kolom Utama	Fungsi dalam Sistem
Data Destinasi Wisata	56 Destinasi	id, nama_wisata, kategori, kota, alamat, deskripsi, harga_tiket, gambar	Berperan sebagai basis pengetahuan (<i>knowledge base</i>) yang diekstraksi menjadi vektor semantik oleh model all-MiniLM-L6-v2 pada metode <i>Content-Based Filtering</i> .

Jenis Dataset	Jumlah Data	Atribut / Kolom Utama	Fungsi dalam Sistem
Data <i>Implicit Feedback</i>	1.185 rekaman klik dari 100 pengguna simulasi	id, user_id, wisata_id, wisata_name, timestamp	Berperan ganda, sebagai kueri teks agregasi dinamis pada metode <i>Content-Based Filtering</i> , sekaligus ditransformasikan menjadi matriks probabilitas untuk metode <i>Collaborative Filtering</i> .

Secara keseluruhan, Data Destinasi Wisata berfungsi sebagai basis pengetahuan (*knowledge base*) yang ditransformasikan menjadi representasi vektor dan disimpan di dalam ChromaDB. Sementara itu, Data *Implicit Feedback* berfungsi sebagai kueri dinamis yang memicu mesin komputasi untuk mencari destinasi mana yang paling relevan dengan riwayat klik pengguna pada saat itu, baik melalui perhitungan jarak kedekatan vektor maupun kemiripan pola interaksi matriks pengguna.

4.1.3 Implementasi Fitur Rekomendasi

Sebagai inti dari kecerdasan buatan sistem JemberTrip, sisi peladen (*backend*) diimplementasikan untuk menjalankan tiga skenario fitur rekomendasi secara bersamaan. Untuk menghindari kelelahan visual (*cognitive load*) pada pembaca, rincian perbedaan alur komputasi, parameter algoritma, dan jenis kueri dari ketiga fitur tersebut dirangkum secara komparatif pada Tabel 4.5 berikut:

Tabel 4. 5 Fitur Rekomendasi

Nama Fitur	Metode Algoritma	Basis Komputasi	Parameter
Wisata Serupa	<i>Content-Based Filtering</i>	Teks destinasi wisata yang sedang dibuka (<i>Vector Embeddings</i> via ChromaDB).	KNN (K=10).
Rekomendasi Personalize	<i>Collaborative Filtering</i>	Data <i>Implicit Feedback</i> (User-Item Matrix frekuensi klik historis pengguna).	KNN (K=30).
Mungkin Yang Anda Sukai	<i>Hybrid Filtering</i>	Skor mentah probabilitas hasil keluaran dari metode <i>Content-Based Filtering</i> dan <i>Collaborative Filtering</i> .	<i>Min-Max Normalization</i> dilanjutkan perhitungan pembobotan ganda

Nama Fitur	Metode Algoritma	Basis Komputasi	Parameter
			dengan $\alpha = 0.6$ (60% CF, 40% CBF).

Secara operasional, eksekusi algoritma *Content-Based Filtering* berfokus murni pada komputasi ruang vektor statis tanpa melibatkan pola interaksi pengguna lainnya. Sebaliknya, implementasi *Memory-Based Collaborative Filtering* dieksekusi secara dinamis dengan menarik data *clickstream* dari PostgreSQL untuk membentuk matriks frekuensi klik berdimensi dua. *Backend* Python kemudian menghitung kemiripan selera pengguna yang sedang login terhadap seluruh pengguna lain di dalam matriks. Berdasarkan hasil pengujian parameter yang akan dipaparkan lebih lanjut pada subbab 4.3, algoritma ini diimplementasikan dengan parameter *K-Nearest Neighbors* berukuran $K=30$. Sistem menyaring 30 pengguna dengan selera paling mirip, lalu menghitung prediksi destinasi menggunakan rumus *Weighted Sum*.

Sebagai tahap final, implementasi *Hybrid Filtering* diaktifkan untuk menutupi kelemahan *Memory-Based Collaborative Filtering* (yakni masalah pengguna baru). Sistem mengambil skor prediksi mentah dari *Content-Based Filtering* dan *Memory-Based Collaborative Filtering*, menyeragamkan rentang nilainya (0 hingga 1) menggunakan *Min-Max Normalization*, dan menggabungkannya (*Weighted Combination*). Sesuai dengan hasil evaluasi skenario yang membuktikan tingkat akurasi prediksi tertinggi, sistem menginjeksi bobot $\alpha 0.6$. Hasil perhitungan akhir ini meloloskan 6 rekomendasi terbaik (Top-6) yang telah melewati penyaringan eksklusif untuk didistribusikan ke *frontend* (Adapun source code komputasi keseluruhan fitur ini selengkapnya dapat dilihat pada Lampiran X).

4.1.4 Antarmuka Sistem (*User Interface*)

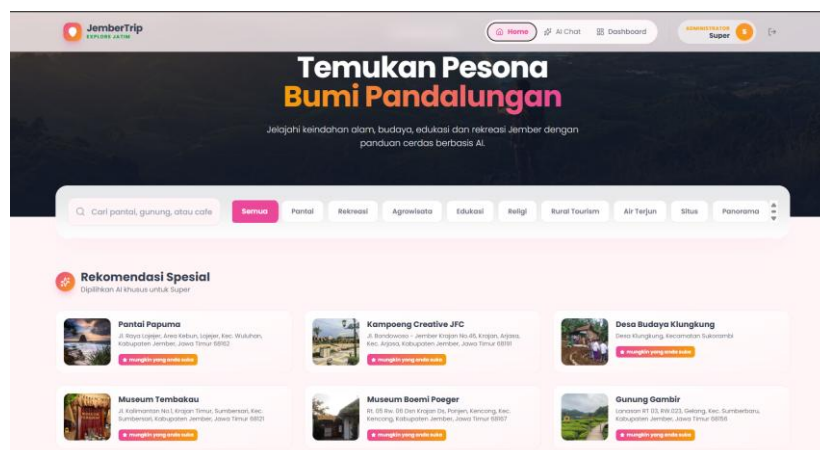
Antarmuka sistem (*frontend*) dibangun sebagai media interaksi utama antara wisatawan dengan mesin kecerdasan buatan di sisi peladen. Antarmuka ini dirancang secara responsif dan bertugas untuk memvisualisasikan hasil komputasi akhir dari ketiga metode algoritma yang telah diterapkan. Implementasi antarmuka pada sistem JemberTrip berfokus pada panel-panel khusus yang menjadi muara dari

keluaran algoritma. Rincian fungsionalitas dari antarmuka ketiga fitur tersebut dirangkum pada Tabel 4.6 berikut:

Tabel 4. 6 Implementasi Antarmuka Fitur Rekomendasi

Fitur	Metode	Keterangan UI	Rujukan Visual
Rekomendasi Personalize	<i>Collaborative Filtering</i>	Menyajikan 6 destinasi wisata (Top-6) yang dipersonalisasi khusus berdasarkan kemiripan pola klik antar-pengguna (User-to-Item). Panel ini muncul di Halaman Beranda.	Gambar 4. 3
Mungkin Yang Anda Sukai	<i>Hybrid Filtering</i>	Tampil berdampingan di bawah panel sebelumnya pada Halaman Beranda, menyajikan 6 destinasi wisata (Top-6) dengan tingkat akurasi tebakan paling tinggi (bobot α 0.6).	Gambar 4. 4
Wisata Serupa	<i>Content-Based Filtering</i>	Menyajikan 3 destinasi alternatif (Top-3) yang memiliki tingkat kemiripan semantik tertinggi (Item-to-Item). Panel ini muncul di bagian bawah Halaman Detail Wisata.	Gambar 4. 5

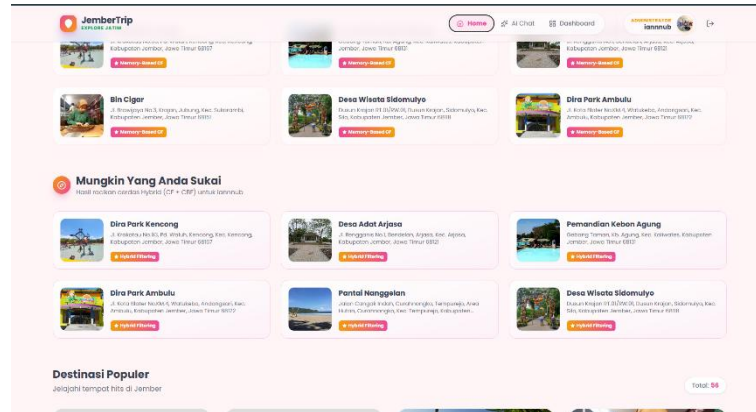
Secara visual, wujud antarmuka dari ketiga fitur rekomendasi yang telah dijelaskan pada Tabel 4.6 di atas dapat dilihat secara berturut-turut pada Gambar 4.3, Gambar 4.4, dan Gambar 4.5 berikut:



Gambar 4. 3 Tampilan Rekomendasi Personalize

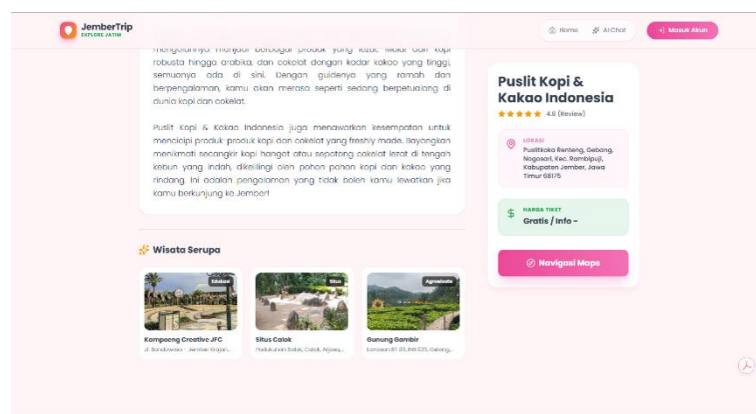
Berdasarkan Gambar 4.3, fitur "Rekomendasi Personalize" ditempatkan secara strategis pada halaman beranda utama (Home). Antarmuka ini menampilkan daftar destinasi yang diproses murni menggunakan algoritma Memory-Based

Collaborative Filtering (CF). Jajaran wisata yang muncul di layar ini merupakan hasil kalkulasi kedekatan *User-Item Matrix* (pola klik), yang secara personal menembak dan memprediksi wisata apa yang kemungkinan besar akan diklik oleh pengguna tersebut di masa depan berdasarkan selera orang lain yang mirip dengannya.



Gambar 4. 4 Tampilan Halaman Yang Mungkin
Anda Sukai

Gambar 4.4 menampilkan wujud antarmuka dari fitur "Mungkin Yang Anda Sukai" yang posisinya berada tepat di bawah panel *Personalize*. Komponen visual ini merupakan muara (output) dari eksekusi algoritma *Hybrid Filtering*. Daftar wisata yang tersaji pada komponen ini memiliki tingkat relevansi yang jauh lebih tinggi dan adaptif, karena merupakan hasil penyatuan skor dari pola klik dinamis (CF) dengan pembobotan *Alpha* yang ditopang oleh perhitungan kemiripan semantik teks statis (CBF).



Gambar 4. 5 Tampilan Halaman Wisata Serupa

Terakhir, Gambar 4.5 mengilustrasikan komponen antarmuka "Wisata Serupa" yang sengaja diletakkan secara eksklusif pada Halaman Detail (ketika

pengguna sedang membaca informasi spesifik suatu destinasi). Visualisasi ini digerakkan secara penuh oleh algoritma *Content-Based Filtering* (CBF) secara *Item-to-Item*. Ketika pengguna membaca halaman "Pantai Papuma", algoritma *Cosine Similarity* akan secara *real-time* menghitung kedekatan vektor teks destinasi lain dan merender daftar wisata alternatif yang memiliki rumpun kategori dan deskripsi yang serupa pada panel ini.

4.2 Skenario Pengujian

Skenario pengujian yang sebelumnya telah didesain secara teoritis pada Bab 3, kini dieksekusi secara komputasional pada lingkungan perangkat lunak menggunakan bahasa pemrograman Python. Untuk memastikan validitas dan objektivitas metrik yang dihasilkan, seluruh skenario dijalankan menggunakan pustaka Scikit-Learn dengan menerapkan modul validasi silang K-Fold ($n_splits=10$). Secara garis besar, pengujian komputasional ini mengevaluasi kinerja prediksi dengan cara membandingkan daftar 6 rekomendasi teratas (Top-6) yang dikeluarkan oleh mesin kecerdasan buatan, untuk dicocokkan terhadap *Ground Truth*.

4.2.1 Skenario Pengujian *Content-Based Filtering* (Wisata Serupa)

Pada skenario ini, pengujian difokuskan pada kemampuan mesin dalam merekomendasikan destinasi alternatif yang memiliki kemiripan semantik. Eksekusi skenario ini dijalankan secara mandiri menggunakan dataset statis destinasi wisata (56 item) tanpa melibatkan dataset *implicit feedback*. Skrip pengujian mengeksekusi algoritma secara berurutan sesuai dengan tahapan pada Tabel 4.7 berikut:

Tabel 4. 7 Rincian Eksekusi Skenario Pengujian *Content-Based Filtering*

No	Tahapan Komputasi	Deskripsi
1	Pemecahan Dataset (<i>Data Splitting</i>)	Menggunakan modul KFold, sistem memecah 56 data destinasi wisata ke dalam 10 lipatan (<i>fold</i>). Pada setiap iterasi, 1 fold diasingkan sebagai <i>test set</i> (kueri uji), sedangkan 9 <i>fold</i> sisanya bertindak sebagai training set (koleksi referensi).
2	Ekstraksi Vektor & Komputasi Jarak	Teks deskriptif dari destinasi yang sedang diuji (<i>test set</i>) dikonversi menjadi matriks numerik menggunakan model all-MiniLM-L6-v2. Vektor uji ini kemudian diukur kedekatannya secara linear terhadap seluruh vektor di dalam <i>training set</i> menggunakan fungsi matematis <i>cosine similarity</i> .

3	Penyortiran KNN dan Pembatasan Top-6	Array hasil komputasi <i>Cosine Similarity</i> diurutkan secara menurun (<i>descending</i>). Algoritma kemudian menerapkan parameter <i>neighbors</i> (<i>K-Nearest Neighbors</i>) untuk menyaring batas kandidat, lalu memotong dan merestriksi hasil akhir agar hanya menyajikan 6 destinasi dengan skor kedekatan tertinggi (Top-6).
4	Validasi <i>Ground Truth</i>	Skrip pengujian mengekstrak atribut "kategori" dari destinasi acuan (<i>test set</i>). Sistem mencatat status Hit (Relevan) apabila terdapat destinasi dari daftar Top-6 yang memiliki label kategori yang sama persis dengan test set.
5	Pencatatan Skor Metrik	Jumlah Hit tersebut kemudian dikalkulasi secara matematis menggunakan rumus untuk menghasilkan nilai akhir <i>Precision</i> , <i>Recall</i> , dan <i>F1-Score</i> pada iterasi tersebut.

4.2.2 Skenario Pengujian *Memory-Based Collaborative Filtering* (Rekomendasi *Personalize*)

Pada skenario ini, pengujian difokuskan pada kemampuan sistem dalam merekomendasikan destinasi secara personal berdasarkan pola kemiripan interaksi antar-pengguna (*User-to-Item*). Eksekusi skenario ini bergantung sepenuhnya pada pengolahan data riwayat klik (*clickstream*). Sama seperti skenario pada 4.2.1, skrip pengujian Python mengeksekusi algoritma secara berurutan sesuai dengan tahapan yang dijabarkan pada Tabel 4.8 berikut:

Tabel 4. 8 Rincian Eksekusi Skenario Pengujian *Memory-Based Collaborative Filtering*

No	Tahapan Komputasi	Deskripsi
1	Pemecahan Dataset (<i>Data Splitting</i>)	Menggunakan modul <i>K-Fold</i> , sistem memecah dataset <i>implicit feedback</i> (1.185 baris riwayat klik) ke dalam 10 lipatan (<i>fold</i>). Pada setiap putaran evaluasi, 1 <i>fold</i> digunakan sebagai data uji (<i>test set</i>), sementara 9 <i>fold</i> sisanya digunakan sebagai data histori latih (<i>training set</i>).
2	Pembentukan Matriks Interaksi	Skrip mengonversi data training set menjadi <i>User-Item Matrix</i> dua dimensi. Baris pada matriks merepresentasikan ID pengguna, kolom merepresentasikan ID destinasi, dan sel matriks diisi dengan nilai frekuensi klik aktual.
3	Komputasi Kemiripan (<i>User-to-User</i>)	Sistem mengeksekusi fungsi <i>cosine similarity</i> secara horizontal antar-baris matriks untuk menghasilkan skor tingkat kemiripan selera di antara seluruh profil pengguna yang ada.
4	Penyaringan & Prediksi (<i>Weighted Sum</i>)	Algoritma <i>K-Nearest Neighbors</i> (KNN) menyaring matriks dan hanya mengambil

		sejumlah pengguna dengan skor kemiripan tertinggi (sejumlah parameter batas K yang sedang dievaluasi). Sistem kemudian menebak probabilitas destinasi yang disukai pengguna dengan mengalikan matriks frekuensi klik terhadap skor kemiripan tetangga-tetangga tersebut (<i>Weighted Sum</i>).
5	Pembatasan Keluaran (Top-6)	Sistem secara ketat menghapus destinasi yang sudah ada di <i>training set</i> pengguna (agar tidak merekomendasikan tempat yang sudah dikunjungi), lalu mengambil 6 destinasi dengan skor tebakan tertinggi (Top-6).
6	Validasi <i>Ground Truth</i> & Metrik	Sistem mencocokkan hasil rekomendasi Top-6 dengan data yang disembunyikan di dalam test set. Apabila tebakan destinasi tersebut benar-benar ada di dalam <i>test set</i> , sistem mencatatnya sebagai Hit. Angka ketepatan ini dikalkulasi menjadi nilai akhir <i>Precision</i> , <i>Recall</i> , dan <i>F1-Score</i> .

4.2.3 Skenario Pengujian *Hybrid Filtering (Weighted Hybrid)*

Skenario ini merupakan puncak pengujian untuk membuktikan bahwa metode gabungan memiliki performa yang lebih stabil dan sanggup menutupi kelemahan yang ada pada masing-masing algoritma tunggal. Skenario ini dijalankan bersamaan dengan putaran evaluasi *K-Fold* pada skenario sebelumnya. Skrip pengujian mengeksekusi algoritma *Hybrid* ini sesuai dengan tahapan matematis yang dijabarkan pada Tabel 4.9 berikut:

Tabel 4. 9 Rincian Eksekusi Skenario Pengujian *Hybrid Filtering*

No	Tahapan Komputasi	Deskripsi
1	Pengumpulan Skor Mentah (<i>Raw Score</i>)	Pada setiap iterasi pengujian, sistem mengekstrak daftar skor prediksi mentah yang baru saja dihasilkan oleh perhitungan <i>Content-Based Filtering</i> (berbasis jarak vektor) dan <i>Collaborative Filtering</i> (berbasis matriks probabilitas).
2	Penyeragaman Skala (<i>Min-Max Normalization</i>)	Karena kedua algoritma menghasilkan skor dengan rentang matematis yang berbeda, skrip mengeksekusi rumus <i>Min-Max Normalization</i> menggunakan pustaka NumPy untuk memaksa seluruh skor mentah dari kedua algoritma berada pada rentang rasio desimal yang setara, yakni dari angka 0 hingga 1.
3	Penggabungan Pembobotan (<i>Alpha</i>)	Skor CBF dan CF yang telah seragam tersebut kemudian dijumlahkan secara matematis menggunakan rumus <i>Weighted Combination</i> . Skrip menginjeksi variabel pembobotan secara dinamis dalam rentang iterasi <i>Alpha</i> 0.1 hingga 0.9

	(untuk mencari parameter terbaik), di mana besaran <i>Alpha</i> menjadi porsi pengali untuk skor CF, dan sisa porsinya ($1 - Alpha$) untuk skor CBF guna mendapatkan nilai kecocokan <i>Hybrid</i> akhir.
4	Penyortiran & Pembatasan Keluaran (Top-6)
5	Validasi <i>Ground Truth</i> & Metrik Akhir

4.3 Hasil Evaluasi dan Pembahasan

Bagian ini merupakan inti pembuktian dari penelitian, di mana seluruh skenario pengujian yang telah dijabarkan sebelumnya dieksekusi secara nyata untuk mendapatkan nilai kuantitatif dari metrik evaluasi. Hasil yang diperoleh dari komputasi ini tidak hanya membuktikan secara objektif mengenai performa masing-masing metode rekomendasi, tetapi juga memberikan justifikasi ilmiah (pencarian *hyperparameter*) mengapa sistem menetapkan batas nilai *K-Nearest Neighbors* dan bobot parameter *Alpha* tertentu ke dalam implementasi final perangkat lunak. Pembahasan hasil evaluasi ini dijabarkan secara bertahap, dimulai dari komputasi algoritma tunggal hingga analisis komparatif pada algoritma *Hybrid*.

4.3.1 Hasil Evaluasi Skenario *Content-Based Filtering* (CBF)

Skenario pertama mengevaluasi seberapa akurat metode *Content-Based Filtering* (CBF) dalam merekomendasikan destinasi wisata serupa (*Item-to-Item*) berdasarkan kemiripan deskripsi teks. Sebelum mengekstraksi nilai metrik akhir, pengujian diawali dengan tahap pencarian parameter (*Hyperparameter Tuning*) untuk menentukan batasan *K-Nearest Neighbors* (KNN) yang paling optimal.

Skrip pengujian mengeksekusi skenario validasi 10-Fold secara berulang dengan menaikkan rentang parameter K pada kelipatan 5 (dari 10 hingga 30). Hasil keluaran metrik dari pengujian tersebut disajikan pada Tabel 4.10 berikut:

Tabel 4. 10 Hasil Pengujian Parameter K pada *Content-Based Filtering*

Nilai KNN (K)	<i>Precision@6</i>	<i>Recall@6</i>	<i>F1-Score@6</i>
K = 10	56.55%	56.24%	52.21%
K = 15	56.55%	56.24%	52.21%
K = 20	56.55%	56.24%	52.21%
K = 25	56.55%	56.24%	52.21%
K = 30	56.55%	56.24%	52.21%

Berdasarkan Tabel 4.10, dapat diamati bahwa manipulasi terhadap rentang nilai K tidak memberikan fluktuasi sedikit pun pada hasil akurasi CBF. Hal ini secara logis disebabkan oleh sifat algoritma CBF yang mencari kemiripan secara statis dan langsung antar-vektor teks (murni *Item-to-Item*), sehingga berapapun batas radius K yang diberikan, urutan kedekatan semantik destinasi (*Cosine Similarity*) tidak akan pernah berubah. Atas dasar efisiensi komputasi agar memori peladen (*server*) tidak perlu memuat matriks *neighbors* yang terlalu panjang, maka sistem menetapkan parameter terkecil yakni K=10 sebagai batas pencarian final untuk fitur ini.

Dengan ditetapkan parameter K=10, pengujian Skenario 1 mencatatkan metrik performa yang sangat tinggi, dengan nilai ***Precision* 56.55%**, ***Recall* 56.24%**, dan ***F1-Score* 52.21%**. Tingginya persentase ini membuktikan bahwa arsitektur model Transformer all-MiniLM-L6-v2 sangat tangguh dalam memahami konteks semantik. Algoritma ini terbukti sukses (Hit) dalam menjejalkan destinasi wisata alternatif dengan kategori yang sama persis (misal: wisata pantai direkomendasikan bersama wisata pantai lainnya) ke dalam daftar rekomendasi Top-6 (Bukti visual cuplikan layar hasil komputasi metrik pada Terminal dapat dilihat secara utuh pada Lampiran X).

4.3.2 Hasil Evaluasi Skenario Memory-Based *Collaborative Filtering*

Skenario kedua bertujuan menguji keakuratan metode Memory-Based *Collaborative Filtering* (CF) dalam menebak destinasi apa yang akan diklik oleh seorang pengguna secara personal, murni berdasarkan kedekatan User-Item Matrix antar-pengguna. Sama halnya dengan skenario sebelumnya, pengujian diawali dengan tahap pencarian parameter K-Nearest Neighbors (KNN).

Pada *Collaborative Filtering*, nilai K mendefinisikan seberapa banyak jumlah *nearest neighbors* (pengguna lain yang mirip) yang akan dilibatkan ke dalam rumus *Weighted Sum* untuk memprediksi rekomendasi. Hasil keluaran 10-

Fold Cross Validation terhadap rentang K=10 hingga K=30 disajikan pada Tabel 4.11 berikut:

Tabel 4. 11 Hasil Pengujian Parameter K pada *Collaborative Filtering*

Nilai KNN (K)	<i>Precision@6</i>	<i>Recall@6</i>	<i>F1-Score@6</i>
K = 10	6.90%	28.82%	10.70%
K = 15	7.13%	28.78%	11.01%
K = 20	7.11%	28.41%	10.93%
K = 25	7.22%	29.06%	11.14%
K = 30	7.27%	29.02%	11.19%

Berdasarkan Tabel 4.11, terlihat jelas bahwa penambahan jumlah tetangga memberikan pengaruh yang fluktuatif namun cenderung positif terhadap performa tebakan algoritma. Pelibatan 30 pengguna dengan selera termirip terbukti memberikan keseimbangan harmonis (*F1-Score*) terbaik. Oleh karena itu, sistem menetapkan parameter K=30 sebagai batas algoritma KNN pada fitur Rekomendasi Personalize.

Dengan berbekal K=30, skenario CF menghasilkan nilai *Precision* 7.27%, *Recall* 29.02%, dan metrik final *F1-Score* sebesar 11.19%. Terlihat adanya jarak persentase akurasi yang cukup lebar jika disandingkan secara mentah dengan hasil evaluasi *Content-Based Filtering* (52.21%). Namun, hal ini secara alamiah disebabkan oleh perbedaan tingkat kesulitan prediksi (task complexity). Evaluasi CBF secara komputasional lebih mudah karena hanya bertugas "mencocokkan atribut" (mencari wisata dengan kategori yang sama).

Sebaliknya, evaluasi algoritma CF memiliki tantangan yang jauh lebih berat karena diwajibkan untuk menebak secara eksak "destinasi rahasia mana yang kelak akan diklik oleh pengguna di masa depan" di tengah kondisi dataset clickstream yang sangat acak dan memiliki rasio kekosongan sel yang ekstrem (data sparsity problem). Dalam ranah literatur sistem rekomendasi implicit feedback, pencapaian *F1-Score* yang menembus angka belasan persen pada dataset bervolume rendah seperti ini mengindikasikan bahwa perhitungan matriks Cosine Similarity pada algoritma CF telah berfungsi dengan baik dan rasional (Bukti visual cuplikan layar hasil komputasi Terminal untuk CF dapat dilihat pada Lampiran 8).

4.3.3 Hasil Evaluasi Skenario *Hybrid Filtering*

Skenario ketiga dan terakhir ini dieksekusi guna mengevaluasi sejauh mana penggabungan (*Hybrid*) antara skor probabilitas CBF dan CF mampu

meningkatkan akurasi tebakan destinasi. Pengujian skenario ini diawali dengan tahap penyeragaman skala nilai skor (*Min-Max Normalization*) sebagaimana merujuk pada Persamaan (1), agar kedua metode tersebut dapat digabungkan secara proporsional.

Sebelum melakukan pencarian pembobotan parameter *Alpha* menggunakan Persamaan (2), penelitian ini terlebih dahulu melakukan pencarian parameter *K-Nearest Neighbors* (KNN) yang paling optimal khusus untuk kerangka kerja *Hybrid Filtering*. Pengujian K ini dilakukan menggunakan nilai *Alpha* penyeimbang (0.5). Hasil pengujian parameter K disajikan pada Tabel 4.12 berikut:

Tabel 4. 12 Hasil Pengujian Pengujian Parameter K pada *Hybrid Filtering*

Nilai KNN (K)	<i>Precision@6</i>	<i>Recall@6</i>	<i>F1-Score@6</i>
K = 10	7.58%	30.83%	11.73%
K = 15	7.74%	31.53%	11.99%
K = 20	7.83%	31.82%	12.12%
K = 25	7.86%	31.73%	12.14%
K = 30	7.92%	32.12%	12.25%

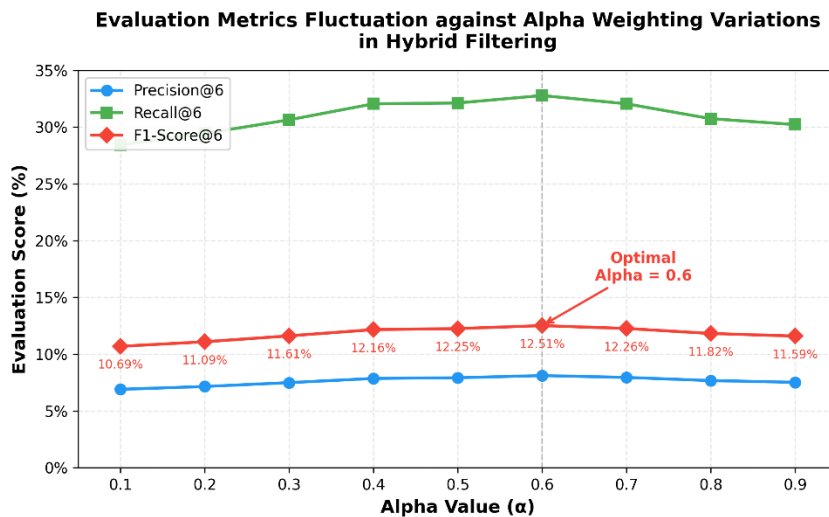
Berdasarkan Tabel 4.12, terbukti bahwa nilai tetangga K=30 menghasilkan performa akurasi tertinggi (*F1-Score 12.25%*). Oleh karena itu, nilai K=30 ini dikunci dan dijadikan standar dasar untuk tahapan selanjutnya, yaitu pencarian pembobotan parameter *Alpha* terbaik.

Pada skema *Weighted Hybrid*, rentang nilai *Alpha* dieksplorasi secara bertahap mulai dari 0.1 hingga 0.9. Semakin besar nilai *Alpha*, maka tebakan rekomendasi akan semakin condong dipengaruhi oleh algoritma CF (pola klik historis). Sebaliknya, semakin kecil nilai *Alpha*, maka rekomendasi akan lebih bergantung pada algoritma CBF (kemiripan teks destinasi). Hasil pengujian skenario pembobotan ini disajikan pada Tabel 4.13 berikut:

Tabel 4. 13 Hasil Pengujian Pembobotan *Alpha* pada *Hybrid Filtering*

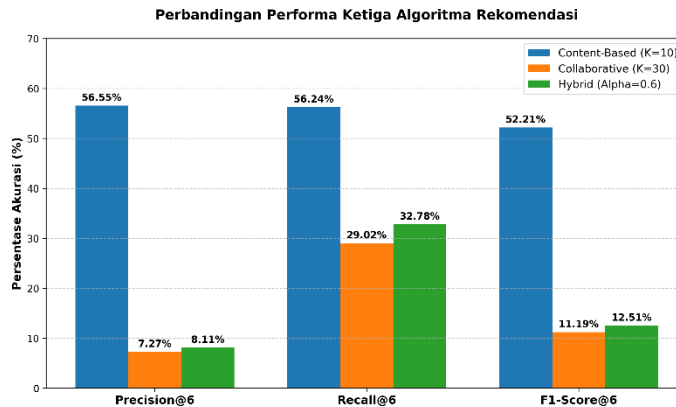
Pembobotan (<i>Alpha</i>)	<i>Precision@6</i>	<i>Recall@6</i>	<i>F1-Score@6</i>
<i>Alpha</i> 0.1	6.90%	28.44%	10.69%
<i>Alpha</i> 0.2	7.15%	29.44%	11.09%
<i>Alpha</i> 0.3	7.49%	30.64%	11.61%
<i>Alpha</i> 0.4	7.86%	32.05%	12.16%
<i>Alpha</i> 0.5	7.92%	32.12%	12.25%
<i>Alpha</i> 0.6	8.11%	32.78%	12.51%
<i>Alpha</i> 0.7	7.95%	32.05%	12.26%
<i>Alpha</i> 0.8	7.67%	30.74%	11.82%
<i>Alpha</i> 0.9	7.52%	30.22%	11.59%

Berdasarkan fluktuasi kurva nilai pada Tabel 4.13 serta sebagaimana ditunjukkan secara visual pada Gambar 4.6, dapat diamati bahwa injeksi parameter *Alpha* 0.6 berhasil menembus puncak performa paling optimal dengan menorehkan nilai ***Precision* 8.11%**, ***Recall* 32.78%** dan ***F1-Score* 12.51%**. Angka 0.6 ini secara harfiah berarti sistem mengadopsi komposisi 60% dari tebakan CF dan 40% dari tebakan CBF. Komposisi seimbang ini terbukti paling cerdas, karena mesin kecerdasan buatan JemberTrip pada akhirnya lebih memprioritaskan rekam jejak perilaku clickstream pengguna (karena paling merepresentasikan minat orisinal), namun tetap menggunakan skor kemiripan teks statis (CBF) dengan porsi penyokong (*support factor*) yang cukup besar untuk mengoreksi tebakan jika terjadi masalah data sparsity pada matriks.



Gambar 4. 6 Grafik Fluktuasi Metrik Evaluasi
Terhadap Variasi Bobot *Alpha*

Berdasarkan serangkaian evaluasi panjang yang telah dilakukan, dapat ditarik kesimpulan komparatif dari ketiga metode tersebut. Untuk mempermudah visualisasi perbandingan performa antara *Content-Based Filtering* ($K=10$), *Collaborative Filtering* ($K=30$), dan *Hybrid Filtering* ($Alpha=0.6$), keseluruhan metrik diekstraksi dan dirangkum ke dalam bentuk diagram batang pada Gambar 4.7 berikut:



Gambar 4. 7 Perbandingan Performa Ketiga Algoritma Rekomendasi

Merujuk pada visualisasi Gambar 4.7, terlihat jelas disparitas (perbedaan) performa yang mencolok dari ketiga pendekatan algoritma. Pengujian *Content-Based Filtering* (CBF) secara absolut mendominasi seluruh metrik evaluasi dengan menorehkan nilai tertinggi, yaitu *Precision* mencapai 56,55%, *Recall* 56,24%, dan *F1-Score* 52,21%. Tingginya angka ini membuktikan bahwa arsitektur pencarian vektor semantik murni sangat andal untuk dieksekusi secara statis karena mesin hanya bertugas mencocokkan kemiripan teks antar-destinasi (*Item-to-Item*). Namun, terlepas dari tingginya akurasi tersebut, metode ini memiliki kelemahan fungsional karena bersifat kaku dan tidak mampu menyuguhkan rekomendasi personal lintas-kategori kepada wisatawan.

Di sisi lain, saat sistem diberi beban tugas prediksi yang jauh lebih kompleks yakni menebak pola klik dinamis secara personal (*User-to-Item*), algoritma *Memory-Based Collaborative Filtering* (CF) tunggal mengalami penurunan metrik yang sangat drastis dengan raihan *Precision* 7,27%, *Recall* 29,02%, dan *F1-Score* 11,19%. Merosotnya angka ini merupakan fenomena lumrah (*bottleneck*) pada sistem perekomendasi berbasis *implicit feedback* murni. Algoritma CF kesulitan mencari kedekatan matriks antarpengguna akibat tingginya rasio kekosongan interaksi riwayat (*data sparsity problem*), sehingga mesin gagal menebak destinasi-destinasi baru yang masih minim jejak klik (*cold-start problem*).

Di titik inilah implementasi *Hybrid Filtering* membuktikan taring utamanya. Melalui penyatuan algoritma pola klik (CF) dengan kecerdasan semantik teks (CBF) menggunakan konfigurasi optimal bobot *Alpha* 0.6, akurasi tebakan sistem terbukti sukses terangkat secara konsisten di seluruh lini metrik melampaui

kemampuan algoritma CF tunggal. Rincian peningkatan (*improvement*) performa tersebut dirangkum pada Tabel 4.14 berikut:

Tabel 4. 14 Ringkasan Peningkatan Performa (CF + *Hybrid*)

Metrik Evaluasi	<i>Collaborative Filtering</i> (K=30)	<i>Hybrid Filtering</i> (<i>Alpha</i> 0.6)	Peningkatan Kinerja
<i>Precision@6</i>	7.27%	8.11%	+ 0.84%
<i>Recall@6</i>	29.02%	32.78%	+ 3.76%
<i>F1-Score@6</i>	11.19%	12.51%	+ 1.32%

Berdasarkan Tabel 4.14, terlihat jelas bahwa metode *Hybrid Filtering* berhasil memberikan dorongan akurasi yang konsisten. Nilai *Precision* merangkak naik sebesar 0,84% menjadi 8,11%, *Recall* mendapat suntikan peningkatan terbesar sebanyak 3,76% menjadi 32,78%, dan metrik keseimbangan *F1-Score* secara keseluruhan sukses meningkat 1,32% menjadi 12,51%. Dalam literatur sistem rekomendasi berbasis *implicit feedback* yang memproses rentetan data klik secara acak, lompatan akurasi lebih dari 1% ini merupakan pencapaian tebakan yang sangat signifikan (Bukti visual hasil komputasi metrik pada Terminal disematkan pada Lampiran 10).

Sebagai konklusi final, kehadiran metode *Hybrid Filtering* di dalam sistem rekomendasi JemberTrip terbukti secara empiris dan komputasional mampu menambal celah kelemahan *cold-start problem* dari algoritma tradisional. Injeksi pengetahuan teks berbasis Transformer sukses mendongkrak performa tebakan menjadi lebih akurat dibandingkan hanya mengandalkan metode pola klik tunggal, sehingga wisatawan dapat mengeksplorasi suguhan rekomendasi personal yang jauh lebih variatif dan relevan dengan selera aktual mereka.

BAB 5 PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian komputasional (10-Fold Cross Validation) yang telah dilakukan terhadap sistem rekomendasi wisata JemberTrip, maka dapat ditarik kesimpulan untuk menjawab rumusan masalah sebagai berikut:

1. Tingkat akurasi dari sistem rekomendasi wisata berbasis implicit feedback menggunakan metode murni *Content-Based Filtering* (berbasis kemiripan vektor teks destinasi) menghasilkan metrik performa yang sangat tinggi, yakni dengan nilai *Precision* sebesar 56.55%, *Recall* 56.24%, dan *F1-Score* 52.21% (pada K-Nearest Neighbors K=10). Tingginya akurasi ini membuktikan bahwa metode CBF sangat unggul jika diimplementasikan untuk merekomendasikan daftar wisata alternatif yang memiliki kategori serumpun (fitur "Wisata Serupa" secara Item-to-Item).
2. Tingkat akurasi dari sistem rekomendasi wisata berbasis implicit feedback menggunakan metode Memory-Based *Collaborative Filtering* (berbasis kemiripan User-Item Matrix riwayat klik pengguna) menghasilkan nilai *Precision* sebesar 7.27%, *Recall* 29.02%, dan *F1-Score* 11.19% (pada batas pencarian K=30). Angka akurasi prediksi ini membuktikan bahwa metode CF mampu memberikan rekomendasi personal (fitur "Rekomendasi Personalize" di halaman beranda) secara rasional, meskipun dihadapkan pada tingkat kesulitan prediksi dataset riwayat klik yang sangat acak dan bervolume rendah (data sparsity).
3. Tingkat akurasi dari sistem rekomendasi menggunakan metode *Hybrid Filtering* terbukti berhasil melampaui performa algoritma pola klik tunggal (CF). Melalui penggabungan skor menggunakan *Min-Max Normalization* dan pembobotan *Alpha* 0.6, akurasi metode *Hybrid* (fitur "Mungkin Yang Anda Sukai") meningkat menjadi nilai *Precision* sebesar 8.11%, *Recall* 32.78%, dan *F1-Score* 12.51%. Hal ini menyimpulkan bahwa injeksi algoritma *Hybrid* sukses mengoreksi kelemahan pada algoritma tunggal

sehingga rekomendasi akhir yang diberikan kepada wisatawan menjadi jauh lebih relevan.

5.2 Saran

Berdasarkan serangkaian evaluasi dan batasan masalah yang ditemukan selama pengembangan sistem rekomendasi JemberTrip, terdapat beberapa saran yang dapat diajukan sebagai acuan untuk pengembangan dan penelitian di masa mendatang, yaitu:

1. Penerapan *Model-Based Collaborative Filtering*

Penelitian ini menggunakan pendekatan Memory-Based CF (KNN) yang cukup rentan terhadap masalah kekosongan matriks (*data sparsity*) apabila jumlah wisatawannya terlalu sedikit. Penelitian selanjutnya disarankan untuk menguji coba algoritma *Model-Based CF* seperti *Matrix Factorization* (SVD) atau *Deep Learning (Neural Collaborative Filtering)* untuk mengompresi matriks dan mengekstraksi relasi tersembunyi (*latent factors*) agar prediksi *clickstream* menjadi lebih akurat.

2. Penggabungan dengan *Explicit Feedback*

Meskipun *implicit feedback* (riwayat klik) sangat baik untuk menangkap minat alamiah tanpa mengganggu pengguna, sistem tidak dapat membedakan apakah pengguna mengklik wisata tersebut karena benar-benar tertarik atau sekadar penasaran lalu kecewa (tidak suka). Penelitian ke depan dapat menyempurnakan pembobotan matriks dengan memadukan data *implicit* (klik) dengan fitur *explicit feedback* (seperti tombol "Suka", pemberian Rating Bintang, atau Ulasan Teks).

3. Penerapan *Context-Aware Recommendation System* (CARS)

Sistem yang dibangun saat ini belum mempertimbangkan faktor kondisi lingkungan di dunia nyata. Penelitian selanjutnya dapat mengintegrasikan variabel *Context-Aware* seperti lokasi terkini pengguna (GPS), waktu akses (pagi/sore), serta kondisi cuaca (hujan/cerah). Dengan demikian, rekomendasi yang diberikan akan menjadi lebih logis (contoh: tidak merekomendasikan wisata pantai terbuka saat cuaca sedang hujan deras).

DAFTAR PUSTAKA

- Channarong, C., Paosirikul, C., Maneeroj, S., & Takasu, A. (2022). HybridBERT4Rec : A Hybrid (*Content-Based Filtering* and Collaborative Filtering) Recommender System Based on BERT. *IEEE Access*, 10, 56193–56206. <https://doi.org/10.1109/ACCESS.2022.3177610>
- Dwiputriane, D. B., Abas, Z. A., & Herman, N. S. (2022). Systematic Literature Review on Enhancing Recommendation System by Eliminating Data Sparsity. *Journal of Theoretical and Applied Information Technology*, 100(7), 2254–2270.
- Esmeli, R., Bader, M., Hassana, E. Den, & Henderson, D. (2023). Implicit Feedback Awareness for Session Based Recommendation in E - Commerce. *SN Computer Science*, 4(3), 1–15. <https://doi.org/10.1007/s42979-023-01752-x>
- Faurina, R., & Sitanggang, E. (2023). Implementasi Metode *Content-Based Filtering* dan Collaborative Filtering pada Sistem Rekomendasi Wisata di Bali. *Techno*, 22(4), 870–881.
- Firmansyah, M. F., Aziz, A., & Ahsan, M. (2024). *Peningkatan Kinerja Sistem Rekomendasi Wisata Melalui Penerapan Algoritma Collaborative Filtering Dan K-Nearest Neighbors Dengan Metode Klasterisasi K-Means*. 8(6), 11420–11427.
- George, G., & Lal, A. M. (2022). *Hy-MOM: Hybrid Recommender System Framework Using Memory-Based and Model-Based Collaborative Filtering Framework*. 22(1), 134–150. <https://doi.org/10.2478/cait-2022-0009>
- Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2022). *Knowledge Distillation : A Survey*.
- Gunathilaka, T. M. A. U., Manage, P. D., Zhang, J., Li, Y., & Kelly, W. (2025). Addressing sparse data challenges in recommendation systems: A systematic review of rating estimation using sparse rating data and profile enrichment techniques. *Intelligent Systems with Applications*, 25(December 2024), 200474. <https://doi.org/10.1016/j.iswa.2024.200474>
- Hermanto, B., Parabi, M. I., Sakethi, D., & Azhar, N. (2025). *Performance Comparison of Object-Relational Mapping (ORM) and SQL Query in Developing the Booking Service API at PT Tunas Dwipa Matra*. 6(2), 113–119. <https://doi.org/10.23960/pepadun.v6i2.263>
- Keogh, P. (2025). *A Comparison of Energy Consumption for SQLAlchemy ORM and Plain SQL in CRUD Operations Across Incrementally Scaled Datasets* (Nomor June). Technological University Dublin.
- Khumaidi, M. K. A., Huda, M. M., Dharma, R., & Yusron, R. (2025). *Systematic Literature Review : Analisa Sistem Rekomendasi Tempat Pariwisata Di Kota Palembang*. 6(3), 1328–1338.
- Maskuri, M. N., Harliana, Sukerti, K., & Bhakti, R. M. H. (2022). *Penerapan Algoritma K-Nearest Neighbor (KNN) untuk Memprediksi Penyakit Stroke*. 4(1), 130–140.
- Nangunori, S. K. (2024). *Vector Databases : A Paradigm Shift In High-Dimensional Data Management*. 15(6), 566–577.
- Natasya, R. D. (2023). Implementasi Artificial Intelligence (AI) dalam Teknologi Modern. *Jurnal Komputer dan Teknologi Sains (KOMTEKS)*, 2(1), 22–24.
- Oyadila, S., Abdullah, D., & Razi, A. (2025). *Implementasi Content-Based Filtering Dengan Tf-Idf Dan Cosine Similarity Untuk Sistem Rekomendasi Destinasi*. 10(2), 1329–1339.
- Pratiwi, D., Asrianda, & Rosnita, L. (2024). *Penerapan Metode Content-Based Filtering dalam Sistem Rekomendasi Objek Wisata di Aceh Tamiang*. 4(2), 85–96.
- Rachmaniar, A., Widayati, S., & Rokoyah, K. (2025). *Sistem Rekomendasi Produk E-Commerce Menggunakan Collaborative Filtering dan Content-Based Filtering*. 9(1), 40–54. <https://doi.org/10.52362/jisicom.v9i1.1904>

- Romadhani, L., Putri, Y., & Muhaimin, A. (2025). *Penerapan Metode Content-Based Filtering Berbasis Multilingual Sentence -BERT untuk Sistem Rekomendasi Menu (Studi Kasus : Depot Mie Gemes)*. 01, 82–93.
- Sovina, M., Harahap, F. A., & Lazuly, I. (2024). *Application for Recommending Tourist Attractions on The Island of Java with Content Based Filtering Using Cosine Similarity*. 3(2).
- Sutriawan, Rustad, S., Shidik, G. F., & Pujiono. (2025). *Performance Evaluation of Text Embedding Models for Ambiguity Classification in Indonesian News Corpus: A Comparative Study of TF-IDF, Word2Vec, FastText BERT, and GPT*. 30(6), 1469–1482.
- Suwitno, Wijaya, A. H., & Wiyono. (2025). *Integration Content-Based and Collaborative Filtering in AI-Based Culinary Recommendation For Community of Tangerang City*. 8(2). <https://doi.org/10.32877/bt.v8i2.3182>
- Vaswani, A., Jones, L., Shazeer, N., Gomez, A. N., Parmar, N., Uszkoreit, J., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. *Nips*.
- Wang, H., & Chen, Z. (2026). *Improving Implicit Feedback-Based Recommendation through Multi-Behavior Alignment*. July 2023. <https://doi.org/10.1145/3539618.3591697>
- Wardani, K. R. R., Martina, I., & Wern, J. F. X. (2025). *Analisis Pengaruh Karakteristik Masukan Teks terhadap Kinerja MiniLM v2-L6-H384 dan BERT- Base-Uncased pada Quora Question Pairs*. 20(2), 138–148. <https://doi.org/10.61769/telematika.v20i2.775>
- Winardy, I. C., Fajar, M., & Marlina. (2022). Penerapan Metode Implicit Rating Untuk Sistem Rekomendasi Resep Masakan Berbasis Mobile Web. *Jurnal Ilmu Komputer KHARISMA TECH*, 55–63.

LAMPIRAN

Lampiran 1. Source Code Metode *Content-Based Filtering*

```
# =====  
# FASE 2: CONTENT-BASED FILTERING (SBERT)  
# =====  
u_train = hist_df[hist_df['user_id'] == current_user.id]  
query_text = " ".join(u_train['wisata_name'].tolist())  
q_vec = embedding_model.embed_query(query_text)  
cbf_scores = pd.Series(cosine_similarity([q_vec], sbert_embeddings).flatten(), index=dest_ids)
```

Lampiran 2. Source Code Metode *Memory-Based Collaborative Filtering*

```
# =====  
# FASE 1: MEMORY-BASED CF  
# =====  
all_users = hist_df['user_id'].unique()  
R_train = pd.DataFrame(0.0, index=all_users, columns=dest_ids)  
for (u, i), count in hist_df.groupby(['user_id', 'wisata_id']).size().items():  
    if u in R_train.index and i in R_train.columns:  
        R_train.at[u, i] = float(count)  
  
cf_scores = pd.Series(0.0, index=dest_ids)  
if current_user.id in R_train.index:  
    user_sim_df = pd.DataFrame(cosine_similarity(R_train), index=R_train.index, columns=R_train.index)  
    np.fill_diagonal(user_sim_df.values, 0.0)  
  
    k = 30  
    top_k_users = user_sim_df.loc[current_user.id].nlargest(k).index  
    top_k_sim = user_sim_df.loc[current_user.id, top_k_users]  
    if top_k_sim.max() > 0:  
        cf_scores = R_train.loc[top_k_users].mul(top_k_sim, axis=0).sum(axis=0)
```

Lampiran 3. Source Code Metode *Hybrid Filtering*

```
# =====  
# FASE 3: HYBRID FILTERING  
# ===== You, last week * Uncommitted changes  
cf_norm = (cf_scores - cf_scores.min()) / (cf_scores.max() - cf_scores.min()) if cf_scores.max() > cf_scores.min() else cf_scores * 0.0  
cbf_norm = (cbf_scores - cbf_scores.min()) / (cbf_scores.max() - cbf_scores.min()) if cbf_scores.max() > cbf_scores.min() else  
cbf_scores * 0.0  
  
alpha = 0.7  
hybrid_scores = (alpha * cf_norm) + ((1 - alpha) * cbf_norm)  
  
# Filter tempat yang sudah dikunjungi  
u_train_items = u_train['wisata_id'].unique()  
hybrid_scores = hybrid_scores.drop(index=u_train_items, errors='ignore')  
  
# Ambil 6 Tertinggi  
top_6_recs = hybrid_scores.nlargest(6).index.tolist()  
  
# Format ke bentuk metadata list  
results = [d for d in data_wisata_csv if str(d['id']) in top_6_recs]  
# Urutkan sesuai urutan skor  
results.sort(key=lambda x: top_6_recs.index(str(x['id'])) if str(x['id']) in top_6_recs else 999)  
  
return {"status": "success", "data": results[:6]}  
  
except Exception as e:  
    print(f"Error Hybrid Rek: {e}")  
    return {"status": "success", "data": []}
```

Lampiran 4. Source Code Skrip Pengujian Metrik *Alpha*

```
backend > evaluasi_alpha.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.metrics.pairwise import cosine_similarity
5 from sklearn.model_selection import KFold
6 from sentence_transformers import SentenceTransformer
7 import warnings
8 warnings.filterwarnings('ignore')
9
10 Windsurf: Refactor | Explain | Generate Docstring | X
11 def print_markdown_table(results, headers):
12     print("| " + " | ".join(headers) + " |")
13     print("| " + " | ".join([":---:" for _ in headers]) + " |")
14     for row in results:
15         print("| " + " | ".join([str(row[h]) for h in headers]) + " |")
16
17 # Load Data
18 df_dest = pd.read_csv("data/destinasi_final.csv")
19 df_implicit = pd.read_csv("data/implicit_data_new.csv")
20 df_dest['id'] = df_dest['id'].astype(str)
21 df_implicit['wisata_id'] = df_implicit['wisata_id'].astype(str)
22 df_implicit['timestamp'] = pd.to_datetime(df_implicit['timestamp'])
23 df_implicit = df_implicit.sort_values(by='timestamp')
24
25 df_dest['clean_text'] = (df_dest['nama_wisata'] + " " + df_dest['kategori'] + " " + df_dest['deskripsi']).fillna("")
26 dest_ids = df_dest['id'].tolist()
27
28 print("Loading SBERT Model...")
29 sbert_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
30 sbert_embeddings = sbert_model.encode(df_dest['clean_text'].tolist(), convert_to_numpy=True)
31
32 kf_user = KFold(n_splits=10, shuffle=True, random_state=42)
33 all_users = df_implicit['user_id'].unique()
34 all_items = dest_ids
35
36 backend > evaluasi_alpha.py > ...
37 # =====
38 # PENCARIAN ALPHA TERBAIK - HYBRID FILTERING
39 # =====
40 print("\n" + "="*80)
41 print("=== PENCARIAN ALPHA TERBAIK: HYBRID FILTERING ===")
42 print("=== (Menggunakan K=30, yang merupakan K terbaik dari pengujian CF) ===")
43 print("="*80)
44
45 alpha_values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
46 best_k = 30
47 hybrid_alpha_results = []
48
49 for alpha in alpha_values:
50     precisions, recalls, f1s = [], [], []
51     for train_idx, test_idx in kf_user.split(df_implicit):
52         train_clicks = df_implicit.iloc[train_idx]
53         test_clicks = df_implicit.iloc[test_idx]
54         R_train = pd.DataFrame(0.0, index=all_users, columns=all_items)
55         for (u, i), count in train_clicks.groupby(['user_id', 'wisata_id']).size().items():
56             if u in R_train.index and i in R_train.columns: R_train.at[u, i] = float(count)
57
58         user_sim_df = pd.DataFrame(cosine_similarity(R_train), index=R_train.index, columns=R_train.index)
59         np.fill_diagonal(user_sim_df.values, 0.0)
60
61         for u_id, u_test in test_clicks.groupby('user_id'):
62             if u_id not in user_sim_df.index: continue
63             u_train = train_clicks[train_clicks['user_id'] == u_id]
64             if u_train.empty: continue
65             u_train_items = u_train['wisata_id'].unique()
66
67             # CF
68             top_k_users = user_sim_df.loc[u_id].nlargest(best_k).index
69             top_k_sim = user_sim_df.loc[u_id, top_k_users]
70             cf_scores = R_train.loc[top_k_users].mul(top_k_sim, axis=0).sum(axis=0) if top_k_sim.max() > 0 else pd.Series(0.0, index=all_items)
71
72             # CBF
73             query_text = " ".join(u_train['wisata_name'].tolist())
74             q_vec = sbert_model.encode([query_text], convert_to_numpy=True)
75             cbf_scores = pd.Series(cosine_similarity(q_vec, sbert_embeddings).flatten(), index=dest_ids)
```

```

cf_norm = (cf_scores - cf_scores.min()) / (cf_scores.max() - cf_scores.min()) if cf_scores.max() > cf_scores.min() else cf_scores * 0.0
cbf_norm = (cbf_scores - cbf_scores.min()) / (cbf_scores.max() - cbf_scores.min()) if cbf_scores.max() > cbf_scores.min() else cbf_scores * 0.0

hybrid_scores = (alpha * cf_norm) + ((1 - alpha) * cbf_norm)
hybrid_scores = hybrid_scores.drop(index=u_train_items, errors='ignore')

top_6_recs = hybrid_scores.nlargest(6).index.tolist()
if not top_6_recs or hybrid_scores[top_6_recs[0]] == 0: continue

test_visited_ids = set(u_test['wisata_id'].astype(str).tolist())
hits = sum(1 for rid in top_6_recs if rid in test_visited_ids)
precision = hits / 6.0
recall = hits / len(test_visited_ids)
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

precisions.append(precision)
recalls.append(recall)
f1s.append(f1)

hybrid_alpha_results.append({
    'Alpha (CF Weight)': alpha,
    'Precision@6': f"{np.mean(precisions)*100:.2f}%",
    'Recall@6': f"{np.mean(recalls)*100:.2f}%",
    'F1-Score@6': f"{np.mean(f1s)*100:.2f}%"
})

print_markdown_table(hybrid_alpha_results, ['Alpha (CF Weight)', 'Precision@6', 'Recall@6', 'F1-Score@6'])

```

Lampiran 5. Source Code Skrip Pengujian KNN

```

backend > evaluasi_knn_lengkap.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.metrics.pairwise import cosine_similarity
5 from sklearn.model_selection import KFold
6 from sentence_transformers import SentenceTransformer
7 import warnings
8 warnings.filterwarnings('ignore')
9
10 # Helper functions
11 def calculate_mrr(recs, relevant_set, n):
12     for i, item in enumerate(recs[:n]):
13         if item in relevant_set: return 1.0 / (i + 1)
14     return 0.0
15
16 def calculate_ndcg(recs, relevant_set, n):
17     dcg = sum([1.0 / np.log2(i + 2) for i, item in enumerate(recs[:n]) if item in relevant_set])
18     idcg = sum([1.0 / np.log2(i + 2) for i in range(min(len(relevant_set), n))])
19     return dcg / idcg if idcg > 0 else 0.0
20
21 def print_markdown_table(results, headers):
22     print("| " + " | ".join(headers) + " |")
23     print("| " + " | ".join(["---:" for _ in headers]) + " |")
24     for row in results:
25         print("| " + " | ".join([str(row[h]) for h in headers]) + " |")
26
27 # Load Data
28 df_dest = pd.read_csv("/content/destinasi_final.csv")
29 df_implicit = pd.read_csv("/content/implicit_data_new.csv")
30 df_dest['id'] = df_dest['id'].astype(str)
31 df_implicit['wisata_id'] = df_implicit['wisata_id'].astype(str)
32 df_implicit['timestamp'] = pd.to_datetime(df_implicit['timestamp'])
33 df_implicit = df_implicit.sort_values(by='timestamp')
34
35 df_dest['clean_text'] = (df_dest['nama_wisata'] + " " + df_dest['kategori'] + " " + df_dest['deskripsi']).fillna("")
36 dest_ids = df_dest['id'].tolist()
37 dest_id_to_idx = {idx: i for i, idx in enumerate(dest_ids)}
38

```

```

backend > evaluasi_knn_lengkap.py > ...
77 # =====
78 # 2. EVALUASI KNN TERBAIK - REKOMENDASI SPESIAL (CF)
79 # =====
80 print("\n" + "="*80)
81 print("--- 2. PENCARIAN KNN TERBAIK: REKOMENDASI SPESIAL (CF) ---")
82 print("="*80)
83 cf_results = []
84 for k in k_values:
85     precisions, recalls, f1s = [], [], []
86     for train_idx, test_idx in kf.user.split(df_implicit):
87         train_clicks = df_implicit.iloc[train_idx]
88         test_clicks = df_implicit.iloc[test_idx]
89         R_train = pd.DataFrame(0.0, index=all_users, columns=all_items)
90         for (u, i), count in train_clicks.groupby(['user_id', 'wisata_id']).size().items():
91             if u in R_train.index and i in R_train.columns: R_train.at[u, i] = float(count)
92
93         user_sim_df = pd.DataFrame(cosine_similarity(R_train), index=R_train.index, columns=R_train.index)
94         np.fill_diagonal(user_sim_df.values, 0.0)
95
96         for u_id, u_test in test_clicks.groupby('user_id'):
97             if u_id not in user_sim_df.index: continue
98             top_k_users = user_sim_df.loc[u_id].nlargest(k).index
99             top_k_sim = user_sim_df.loc[u_id, top_k_users]
100             if top_k_sim.max() == 0: continue
101
102             item_scores = R_train.loc[top_k_users].mul(top_k_sim, axis=0).sum(axis=0)
103             u_train_items = train_clicks[train_clicks['user_id'] == u_id]['wisata_id'].unique()
104             item_scores = item_scores.drop(index=u_train_items, errors='ignore')
105
106             top_6_recs = item_scores.nlargest(6).index.tolist()
107             if not top_6_recs or item_scores[top_6_recs[0]] == 0: continue
108
109             test_visited_ids = set(u_test['wisata_id'].astype(str).tolist())
110             hits = sum(1 for rid in top_6_recs if rid in test_visited_ids)
111             precision = hits / 6.0
112             recall = hits / len(test_visited_ids)
113             f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
114             precisions.append(precision); recalls.append(recall); f1s.append(f1)
115
116         cf_results.append({'k': k, 'Precision@6': f"{np.mean(precisions)*100:.2f}%", 'Recall@6': f"{np.mean(recalls)*100:.2f}%", 'F1-Score@6': f"{np.mean(f1s)*100:.2f}%"})
117
118 backend > evaluasi_knn_lengkap.py > ...
118 print_markdown_table(cf_results, ['K', 'Precision@6', 'Recall@6', 'F1-Score@6'])
119
120 # =====
121 # 3. EVALUASI KNN TERBAIK - HYBRID FILTERING
122 # =====
123 print("\n" + "="*80)
124 print("--- 3. PENCARIAN KNN TERBAIK: HYBRID FILTERING (Alpha Default = 0.5) ---")
125 print("="*80)
126 hybrid_results = []
127 for k in k_values:
128     precisions, recalls, f1s = [], [], []
129     for train_idx, test_idx in kf.user.split(df_implicit):
130         train_clicks = df_implicit.iloc[train_idx]
131         test_clicks = df_implicit.iloc[test_idx]
132         R_train = pd.DataFrame(0.0, index=all_users, columns=all_items)
133         for (u, i), count in train_clicks.groupby(['user_id', 'wisata_id']).size().items():
134             if u in R_train.index and i in R_train.columns: R_train.at[u, i] = float(count)
135
136         user_sim_df = pd.DataFrame(cosine_similarity(R_train), index=R_train.index, columns=R_train.index)
137         np.fill_diagonal(user_sim_df.values, 0.0)
138
139         for u_id, u_test in test_clicks.groupby('user_id'):
140             if u_id not in user_sim_df.index: continue
141             u_train = train_clicks[train_clicks['user_id'] == u_id]
142             if u_train.empty: continue
143             u_train_items = u_train['wisata_id'].unique()
144
145             # CF
146             top_k_users = user_sim_df.loc[u_id].nlargest(k).index
147             top_k_sim = user_sim_df.loc[u_id, top_k_users]
148             cf_scores = R_train.loc[top_k_users].mul(top_k_sim, axis=0).sum(axis=0) if top_k_sim.max() > 0 else pd.Series(0.0, index=all_items)
149
150             # CBF
151             query_text = " ".join(u_train['wisata_name'].tolist())
152             q_vec = sbert_model.encode([query_text], convert_to_numpy=True)
153             cbf_scores = pd.Series(cosine_similarity(q_vec, sbert_embeddings).flatten(), index=dest_ids)
154
155             cf_norm = (cf_scores - cf_scores.min()) / (cf_scores.max() - cf_scores.min()) if cf_scores.max() > cf_scores.min() else cf_scores * 0.0
156             cbf_norm = (cbf_scores - cbf_scores.min()) / (cbf_scores.max() - cbf_scores.min()) if cbf_scores.max() > cbf_scores.min() else cbf_scores * 0.0

```



```

157
158         hybrid_scores = (0.5 * cf_norm) + (0.5 * cbf_norm)
159         hybrid_scores = hybrid_scores.drop(index=u_train_items, errors='ignore')
160
161         top_6_recs = hybrid_scores.nlargest(6).index.tolist()
162         if not top_6_recs or hybrid_scores[top_6_recs[0]] == 0: continue
163
164         test_visited_ids = set(u_test['wisata_id'].astype(str).tolist())
165         hits = sum(1 for rid in top_6_recs if rid in test_visited_ids)
166         precision = hits / 6.0
167         recall = hits / len(test_visited_ids)
168         f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
169         precisions.append(precision); recalls.append(recall); f1s.append(f1)
170
171         hybrid_results.append({'K': k, 'Precision@6': f"{np.mean(precisions)*100:.2f}%", 'Recall@6': f"{np.mean(recalls)*100:.2f}%", 'F1-Score@6':
172                               f"{np.mean(f1s)*100:.2f}%"})
173
174     print_markdown_table(hybrid_results, ['K', 'Precision@6', 'Recall@6', 'F1-Score@6'])

```

Lampiran 6. Source Code Skrip Metrik Evaluasi

```

backend > evaluasi_metriks.py > ...
1  import pandas as pd
2  import numpy as np
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.metrics.pairwise import cosine_similarity
5  from sklearn.model_selection import KFold
6  from sentence_transformers import SentenceTransformer
7  import warnings
8  warnings.filterwarnings('ignore')
9
10 # Helper functions
11 def print_markdown_table(results, headers):
12     print("| " + " | ".join(headers) + " |")
13     print("| " + " | ".join([":--:" for _ in headers]) + " |")
14     for row in results:
15         print("| " + " | ".join([str(row[h]) for h in headers]) + " |")
16
17 # Load Data
18 df_dest = pd.read_csv("data/destinasi_final.csv")
19 df_implicit = pd.read_csv("data/implicit_data_new.csv")
20 df_dest['id'] = df_dest['id'].astype(str)
21 df_implicit['wisata_id'] = df_implicit['wisata_id'].astype(str)
22 df_implicit['timestamp'] = pd.to_datetime(df_implicit['timestamp'])
23 df_implicit = df_implicit.sort_values(by='timestamp')
24
25 df_dest['clean_text'] = (df_dest['nama_wisata'] + " " + df_dest['kategori'] + " " + df_dest['deskripsi']).fillna("")
26 dest_ids = df_dest['id'].tolist()
27 dest_id_to_idx = {idx: i for i, idx in enumerate(dest_ids)}
28
29 print("Loading SBERT Model...")
30 sbert_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
31 sbert_embeddings = sbert_model.encode(df_dest['clean_text'].tolist(), convert_to_numpy=True)
32
33 kf = KFold(n_splits=10, shuffle=True, random_state=42)
34 kf_user = KFold(n_splits=10, shuffle=True, random_state=42)
35 all_users = df_implicit['user_id'].unique()
36 all_items = dest_ids

```

```

backend > 🚀 evaluasi_metriks.py > ...
38 # PARAMETER TERBAIK (Diperoleh dari evaluasi_knn.py dan evaluasi_alpha.py)
39 BEST_K_CBF = 10
40 BEST_K_CF = 30
41 BEST_K_HYBRID = 30
42 BEST_ALPHA_HYBRID = 0.6
43
44 print("\n" + "="*80)
45 print("=== EVALUASI METRIKS FINAL (PRECISION, RECALL, F1-SCORE) ===")
46 print("=== (Membandingkan ke-3 metode pada Top-6 dengan Parameter Terbaik) ===")
47 print("="*80)
48
49 final_results = []
50
51 # --- 1. EVALUASI CBF ---
52 precisions, recalls, f1s = [], [], []
53 total_tp_cbf, total_fp_cbf, total_fn_cbf = 0, 0, 0
54 for train_idx, test_idx in kf.split(df_dest):
55     train_df = df_dest.iloc[train_idx]
56     test_df = df_dest.iloc[test_idx]
57     sbert_train = sbert_embeddings[train_idx]
58     for _, test_row in test_df.iterrows():
59         q_emb = sbert_embeddings[test_idx[test_row['id']]].reshape(1, -1)
60         sim = cosine_similarity(q_emb, sbert_train).flatten()
61         recs = train_df.iloc[np.argsort(sim)[::-1][:BEST_K_CBF]].head(6)['id'].tolist()
62         relevant_set = set(train_df[train_df['kategori'] == test_row['kategori']]['id'].tolist())
63         hits = sum(1 for item in recs if item in relevant_set)
64
65         tp = hits
66         fp = 6 - hits
67         fn = len(relevant_set) - hits
68         total_tp_cbf += tp
69         total_fp_cbf += fp
70         total_fn_cbf += fn
71
72         precision = hits / 6.0
73         recall = hits / len(relevant_set) if len(relevant_set) > 0 else 0
74         f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
75
76     precisions.append(precision); recalls.append(recall); f1s.append(f1)
77
78 backend > 🚀 evaluasi_metriks.py > ...
119 final_results.append({
120     'Metode': 'Rekomendasi Spesial (CF)', 'Parameter': f"K={BEST_K_CF}",
121     'Precision@6': f"{np.mean(precisions)*100:.2f}%", 'Recall@6': f"{np.mean(recalls)*100:.2f}%",
122     'F1-Score@6': f"{np.mean(f1s)*100:.2f}%"
123 })
124
125 # --- 3. EVALUASI HYBRID ---
126 precisions, recalls, f1s = [], [], []
127 alpha = BEST_ALPHA_HYBRID
128 for train_idx, test_idx in kf.user_split(df_implicit):
129     train_clicks = df_implicit.iloc[train_idx]
130     test_clicks = df_implicit.iloc[test_idx]
131     R_train = pd.DataFrame(0.0, index=all_users, columns=all_items)
132     for (u, i), count in train_clicks.groupby(['user_id', 'wisata_id']).size().items():
133         if u in R_train.index and i in R_train.columns: R_train.at[u, i] = float(count)
134
135     user_sim_df = pd.DataFrame(cosine_similarity(R_train), index=R_train.index, columns=R_train.index)
136     np.fill_diagonal(user_sim_df.values, 0.0)
137
138     for u_id, u_test in test_clicks.groupby('user_id'):
139         if u_id not in user_sim_df.index: continue
140         u_train = train_clicks[train_clicks['user_id'] == u_id]
141         if u_train.empty: continue
142         u_train_items = u_train['wisata_id'].unique()
143
144         # CF
145         top_k_users = user_sim_df.loc[u_id].nlargest(BEST_K_HYBRID).index
146         top_k_sim = user_sim_df.loc[u_id, top_k_users]
147         cf_scores = R_train.loc[top_k_users].mul(top_k_sim, axis=0).sum(axis=0) if top_k_sim.max() > 0 else pd.Series(0.0, index=all_items)
148
149         # CBF
150         query_text = " ".join(u_train['wisata_name'].tolist())
151         q_vec = sbert_model.encode([query_text], convert_to_numpy=True)
152         cbf_scores = pd.Series(cosine_similarity(q_vec, sbert_embeddings).flatten(), index=dest_ids)
153
154         cf_norm = (cf_scores - cf_scores.min()) / (cf_scores.max() - cf_scores.min()) if cf_scores.max() > cf_scores.min() else cf_scores * 0.0
155         cbf_norm = (cbf_scores - cbf_scores.min()) / (cbf_scores.max() - cbf_scores.min()) if cbf_scores.max() > cbf_scores.min() else cbf_scores * 0.0
156
157         hybrid_scores = (alpha * cf_norm) + ((1 - alpha) * cbf_norm)
158         hybrid_scores = hybrid_scores.drop(index=u_train_items, errors='ignore')

```

```

top_6_recs = hybrid_scores.nlargest(6).index.tolist()
if not top_6_recs or hybrid_scores[top_6_recs[0]] == 0: continue

test_visited_ids = set(u_test['wisata_id'].astype(str).tolist())
hits = sum(1 for rid in top_6_recs if rid in test_visited_ids)

precision = hits / 6.0
recall = hits / len(test_visited_ids)
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

precisions.append(precision); recalls.append(recall); f1s.append(f1)

final_results.append([
    'Metode': 'Hybrid Filtering (CBF + CF)', 'Parameter': f"K={BEST_K_HYBRID}, Alpha={BEST_ALPHA_HYBRID}",
    'Precision@6': f"{np.mean(precisions)*100:.2f}%", 'Recall@6': f"{np.mean(recalls)*100:.2f}%",
    'F1-Score@6': f"{np.mean(f1s)*100:.2f}%"
])

print_markdown_table(final_results, ['Metode', 'Parameter', 'Precision@6', 'Recall@6', 'F1-Score@6'])

```

Lampiran 7. Hasil Eksekusi Komputasi *Content-Based Filtering*

```

=====
=== 1. PENCARIAN KNN TERBAIK: WISATA SERUPA (CBF) ===
=====

```

K	Precision@6	Recall@6	F1-Score@6
10	56.55%	56.24%	52.21%
15	56.55%	56.24%	52.21%
20	56.55%	56.24%	52.21%
25	56.55%	56.24%	52.21%
30	56.55%	56.24%	52.21%

Lampiran 8. Hasil Eksekusi Komputasi Memory-Based CF

```

=====
=== 2. PENCARIAN KNN TERBAIK: REKOMENDASI SPESIAL (CF) ===
=====

```

K	Precision@6	Recall@6	F1-Score@6
10	6.90%	28.38%	10.70%
15	7.13%	28.78%	11.01%
20	7.11%	28.41%	10.93%
25	7.22%	29.06%	11.14%
30	7.27%	29.02%	11.19%

Lampiran 9. Hasil Eksekusi Komputasi *Hybrid Filtering*

```

=====
=== 3. PENCARIAN KNN TERBAIK: HYBRID FILTERING (Alpha Default = 0.5) ===
=====

```

K	Precision@6	Recall@6	F1-Score@6
10	7.58%	30.83%	11.73%
15	7.74%	31.53%	11.99%
20	7.83%	31.82%	12.12%
25	7.86%	31.73%	12.14%
30	7.92%	32.12%	12.25%

Lampiran 10. Hasil Eksekusi Pencarian Bobot *Alpha* pada *Hybrid Filtering*

```

=====
=== PENCARIAN ALPHA TERBAIK: HYBRID FILTERING ===
=== (Menggunakan K=30, yang merupakan K Terbaik dari pengujian CF) ===
=====

```

Alpha (CF Weight)	Precision@6	Recall@6	F1-Score@6
:---: :---: :---: :---:			
0.1 6.90% 28.44% 10.69%			
0.2 7.15% 29.44% 11.09%			
0.3 7.49% 30.64% 11.61%			
0.4 7.86% 32.05% 12.16%			
0.5 7.92% 32.12% 12.25%			
0.6 8.11% 32.78% 12.51%			
0.7 7.95% 32.05% 12.26%			
0.8 7.67% 30.74% 11.82%			
0.9 7.52% 30.22% 11.59%			

Lampiran 11. Dataset Implicit Feedback

backend > data >  implicit_data_new.csv

	id	user_id	wisata_id	wisata_name	timestamp
1	1	2	2	Pantai Watu Ulo	2025-12-17 04:13:46.206
2	2	2	2	Pantai Watu Ulo	2025-12-17 04:13:46.272
3	3	2	56	Teluk Love	2025-12-17 04:13:48.030
4	4	2	12	Pantai Papuma	2025-12-17 04:14:47.845
5	5	2	12	Pantai Papuma	2025-12-17 04:14:47.908
6	6	1	2	Pantai Watu Ulo	2025-12-17 05:38:04.780
7	7	1	2	Pantai Watu Ulo	2025-12-17 05:38:04.879
8	8	3	2	Pantai Watu Ulo	2025-12-19 03:47:45.114
9	9	3	2	Pantai Watu Ulo	2025-12-19 03:47:45.121
10	10	3	23	Pantai Nanggalan	2025-12-19 03:47:51.255
11	11	3	2	Pantai Watu Ulo	2025-12-19 03:48:02.780
12	12	3	2	Pantai Watu Ulo	2025-12-19 03:48:02.979
13	13	1	2	Pantai Watu Ulo	2025-12-24 14:23:19.806
14	14	1	2	Pantai Watu Ulo	2025-12-24 14:23:19.822
15	15	1	21	Pantai Payangan	2025-12-24 14:23:41.058
16	16	1	12	Pantai Papuma	2025-12-24 14:23:46.270
17	17	1	26	Pantai Bandialit	2025-12-24 14:23:48.504
18	18	1	4	Bin Cigar	2025-12-24 14:40:24.536
19	19	1	4	Bin Cigar	2025-12-24 14:40:24.603
20	20	1	8	Pemandian Kebon Agung	2025-12-24 14:41:11.825
21	21	1	8	Pemandian Kebon Agung	2025-12-24 14:41:11.832
22	22	1	48	Desa Budaya Klungkung	2025-12-24 14:41:15.478
23	23	1	33	Situs Calok	2025-12-24 14:41:18.167
24	24	1	52	Museum Tembakau	2025-12-24 14:41:20.907
25	25	1	55	Museum Juang Sroedji Sudut Kalisat	2025-12-24 14:41:23.420
26	26	1	12	Pantai Papuma	2025-12-24 16:59:38.249
27	27	1	12	Pantai Papuma	2025-12-24 16:59:38.647
28	28	1	2	Pantai Watu Ulo	2025-12-24 16:59:44.992
29	29	1	2	Pantai Watu Ulo	2025-12-24 16:59:44.993
30	30	1	23	Pantai Nanggalan	2025-12-24 16:59:50.991
31	31	1	26	Pantai Bandialit	2025-12-24 16:59:54.558